

Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Supervisors:

Dr Michael Bertolacci
Associate Professor Edward Cripps

This thesis is presented for the degree of
Bachelor of Philosophy (Honours) of
The University of Western Australia.

Department of Mathematics and Statistics

Abstract

Bayesian inference is rarely available in closed form, and the standard computational response, Markov chain Monte Carlo, must typically be re-run from scratch for every new dataset. Neural Bayes estimators (NBEs), a recent and fast-developing approach to simulation-based inference, avoid this per-dataset cost by amortisation with a neural network. A neural network is trained once, on simulated data, to approximate the Bayes estimator for some parameters under a Bayesian model, after which inference for any new dataset is a single forward pass. This thesis begins from the observation that training such an estimator itself involves a Monte Carlo computation: the training objective is a Monte Carlo estimate of the Bayes risk, and the gradient that drives training is therefore a random quantity with a variance. Taking this view, we apply variance-reduction techniques to the training of the estimator to see whether they can reduce the computational burden of training.

Our first contribution is to show how Rao-Blackwellisation can be used to make training more efficient. Analogous to Gibbs sampling, this method is applicable when some conditional distributions of the full Bayesian joint distribution are available in closed form. When a block of the parameter vector has a tractable conditional posterior, that block is integrated out of the loss analytically rather than sampled; under squared-error loss this amounts to substituting the conditional posterior mean for the sampled target. The resulting training gradient is shown to be the Rao-Blackwellised variant of the standard one, so the law of total variance gives a positive semi-definite gradient-variance gap: the technique can never increase the variance of the training gradient, for any model or architecture. When applied to Bayesian linear regression as a case study, the Rao-Blackwellised estimator attains a lower loss than the standard NBE in every cell of an experimental grid, closes half of the gap to the Gibbs benchmark, and matches the standard NBE at a quarter of the minibatch size. As a contrasting case, the thesis also studies importance sampling, which requires no tractable conditional posterior and instead changes the distribution of simulated training examples before reweighting them back to the original Bayes-risk objective. This greater generality comes without the same guarantee Rao-Blackwellisation had on gradient variance reduction. In an autoregressive experiment, importance sampling gives no accuracy improvement: the standard estimator is already close to the Bayes-optimal reference across most of the prior, and the proposals considered do not target the main source of remaining error. More aggressive reweighting instead lowers the effective sample size and increases noise. The results therefore separate two kinds of variance reduction for NBEs: Rao-Blackwellisation gives a guaranteed reduction in gradient variance when conjugacy is available, while importance

sampling is more broadly applicable but problem-dependent.

Acknowledgements

I would like to thank my supervisors, Dr Michael Bertolacci and Associate Professor Edward Cripps, for their guidance, patience, and encouragement throughout this project. Their insight shaped both the direction of the work and my development as a researcher, and this thesis would not have taken its present form without them.

I am also grateful to Joshua Fry for many helpful discussions and for his assistance with the implementation of the experiments.

Finally, I thank my family and friends for their constant support over the course of the year.

Contents

Abstract	i
Acknowledgements	iii
1 Introduction	1
1.1 Motivation	1
1.2 Related work	2
1.2.1 Simulation-based inference	2
1.2.2 Normalising flows	3
1.2.3 Neural Bayes estimation and amortised inference	3
1.2.4 The simulation cost of likelihood-free inference	3
1.2.5 Variance reduction for stochastic gradients	4
1.3 Contributions	4
1.4 Thesis outline	5
2 Bayesian Inference	6
2.1 Conjugate Models	7
2.1.1 A conjugate example: linear regression	7
2.2 Markov chain Monte Carlo	8
2.2.1 The Metropolis–Hastings algorithm	9
2.2.2 The Gibbs sampler	10
2.3 Case studies	11
2.3.1 Example 1 — the normal model	12
2.3.2 Example 2 — Bayesian linear regression	13
2.3.3 Example 3 — the AR(1) model	13
2.4 The cost of per-dataset inference	15
3 Amortisation and Neural Bayes Estimators	16
3.1 The Bayes estimator framework	17
3.1.1 Loss functions and posterior summaries	17
3.2 Neural Bayes estimators	19
3.2.1 Training by stochastic gradient descent	20
3.3 Neural architectures	21

3.3.1	The DeepSets architecture	21
3.3.2	Recurrent architectures for sequential data	22
3.3.3	Feed-forward architectures for fixed-dimension data	23
3.4	Example 1: estimating normal parameters	23
4	Rao-Blackwellised Loss Functions	25
4.1	Rao-Blackwellising the loss function	25
4.1.1	The Rao-Blackwell theorem	25
4.1.2	Rao-Blackwellising the Bayes risk	26
4.1.3	The squared-error case	27
4.1.4	The training gradients	28
4.1.5	The linear network case	30
4.2	Application to Bayesian linear regression	31
4.2.1	The Bayesian linear regression model	31
4.2.2	Monte Carlo and Rao-Blackwellised training losses	31
4.2.3	Quantifying the variance reduction	31
4.2.4	The Gibbs benchmark	32
4.2.5	Experimental design	33
4.2.6	Results	33
4.3	Plug-in versus Rao-Blackwellised estimators	36
4.4	Discussion	37
5	Importance Sampling	39
5.1	Importance sampling	40
5.2	Application to an AR(1) model	41
5.2.1	The AR(1) model and a recurrent estimator	41
5.2.2	A family of proposals	42
5.2.3	Results	44
5.2.4	The gradient variance, measured directly	45
5.2.5	Why reweighting cannot help here	47
5.3	When importance sampling can help	49
5.4	Discussion	50
6	Discussion and Conclusion	52
6.1	Future work	53
6.2	Conclusion	53
A	Derivations	55
A.1	Detailed balance for Metropolis–Hastings	55
A.2	The optimal importance sampling proposal	55
B	Experimental details	57
B.1	Configuration and protocol	57
B.2	Main grid results	57

B.3	Batch-size sweep	57
B.4	Bayes-optimal floors	59
B.5	Second design matrix	59
B.6	Reproduction	59

Chapter 1

Introduction

1.1 Motivation

Bayesian inference provides a principled framework for reasoning about unknown parameters under uncertainty. Given observed data and a statistical model, it combines prior beliefs with the likelihood to produce a posterior distribution that encodes our knowledge about the parameters after observing the data. However, for all but a small class of so-called conjugate models the posterior is analytically intractable, and inference must proceed by computational approximation.

The dominant approximation is Markov chain Monte Carlo (MCMC), reviewed in Chapter 2, which constructs a Markov chain whose stationary distribution is the posterior and draws samples by simulating it [3, 25]. MCMC is reliable and theoretically well understood: under broad conditions, the chain converges to the exact posterior, and the approximation error is reduced simply by running it for longer. It carries, however, a structural cost. An MCMC chain targets the posterior for one specific dataset. When a new dataset is observed the entire procedure must typically be repeated from scratch, because the chain transfers no information from one inference problem to the next. In settings where the same model must be fitted to hundreds or thousands of distinct datasets, as arises routinely in spatial statistics and the analysis of replicated experiments [26, 27], the total cost scales linearly in the number of problems, with no economy of scale.

Amortised inference takes the opposite trade-off [31]. Rather than solving each inference problem independently, one invests computational effort once to learn a reusable mapping from data to posterior summaries. Inference for any new dataset then costs only a single evaluation of that mapping. A *neural Bayes estimator* (NBE) [26] realises this idea by training a neural network to approximate the Bayes estimator, the function that maps data to the posterior summary minimising a chosen loss function (developed in Chapter 3). Neural networks are typically used here because they are good function approximators, have reliable black-box training methods, and are fast to run in forward mode. The network is trained on data simulated from the model, so the procedure is likelihood-free: it requires only the ability to simulate, not to evaluate

the likelihood. Once trained, the network can quickly generate parameter estimates (such as posterior means) by applying what is effectively just linear algebra to a new dataset – this is known as a forward pass of the network.

The training of an NBE has at its heart a Monte Carlo computation. The network is fitted by minimising an empirical average of the loss over simulated parameter–data pairs, and this empirical average is a Monte Carlo estimate of the population Bayes risk. Its gradient, the quantity that drives training, is likewise a Monte Carlo estimate of the population gradient, and is therefore a random quantity with a variance.

This thesis takes that observation as its starting point. If training an NBE involves Monte Carlo integration, then the techniques developed over decades to reduce the variance of Monte Carlo estimators can be applied not to the inference itself, but to the training of the estimator. A training gradient with lower variance is a better-behaved optimisation signal, and a better-behaved signal should produce a better estimator for the same computational budget.

The primary technique developed here is *Rao-Blackwellisation* (Chapter 4). The Rao-Blackwell theorem states that conditioning an estimator on additional information cannot increase its variance under a convex loss. Applied to the training gradient of an NBE, it implies that if part of the parameter vector has a tractable conditional posterior given the rest, that part can be integrated out of the loss analytically rather than sampled. The resulting training gradient is provably no noisier than the standard one, and is typically much less noisy. The conditional structure this exploits, conditional conjugacy, is the same structure that makes Gibbs sampling possible, and it is common in hierarchical models.

Another variance-reduction technique, *importance sampling*, is also investigated in Chapter 5. Unlike Rao-Blackwellisation, importance sampling does not require a tractable conditional posterior; it only requires simulated training examples to be drawn from a proposal distribution, then reweighted back such that the network still learns the unbiased estimator. This makes it more broadly applicable in principle, but also less automatic. Its benefit depends on the proposal placing simulation effort where the loss, or the training-gradient noise, is actually concentrated. In the autoregressive experiment this condition fails. The estimator is already close to the exact posterior across most of the prior, and the proposal emphasises regions that are not the dominant source of its remaining error. Reweighting therefore does not improve accuracy; when pushed further, it lowers the effective sample size and adds noise.

1.2 Related work

1.2.1 Simulation-based inference

The technique of producing estimators here belongs to the broader category of *simulation-based inference*: the family of methods for fitting models that can be simulated from but whose likelihood is intractable or too expensive to evaluate [8]. The classical representative of this method is approximate Bayesian computation, which compares simulated and observed data through summary statistics and a tolerance and underlies much of the applied likelihood-free literature.

Over the past decade the field has been reshaped by neural networks, which replace hand-crafted summaries and accept–reject schemes with learned mappings between data and inferential targets [31]. The neural approaches are not uniformly superior. A systematic benchmark finds no single best algorithm, and shows that the choice of performance metric and the sample efficiency of the training scheme matter as much as the architecture [16]; but they have made amortised, reusable inference practical for models that were previously out of reach.

1.2.2 Normalising flows

Neural simulation-based methods are often distinguished by what they learn. *Neural posterior estimation* learns the full posterior density, *neural likelihood estimation* learns the likelihood, and *neural ratio estimation* learns a likelihood ratio for model comparison [20]. Each typically represents the learned distribution with a *normalising flow*, a bijective neural transformation of a simple base density whose tractable Jacobian yields an exact likelihood [19, 21]. The BAYESFLOW framework couples such an invertible network with a summary network that learns informative statistics directly from simulations, giving a globally amortised posterior sampler [23, 24]. These methods return a complete posterior, at the cost of training and storing a generative model of it.

1.2.3 Neural Bayes estimation and amortised inference

Neural Bayes estimation, the setting of this thesis, is the most economical point on this spectrum: it learns only a point summary of the posterior, the Bayes estimator under a chosen loss, trading the full distribution of the density-estimation methods for speed and simplicity. The estimator in the form used here is due to Sainsbury-Dale et al. [26], who establish the decision-theoretic framework, the permutation-invariant architecture for replicated data, and applications to weakly identified and highly parameterised models; the approach has since been extended to irregular spatial data with graph neural networks [27]. It sits within the broader programme of amortised inference surveyed by Zammit-Mangion et al. [31]: methods that pay a one-off training cost up front and thereafter return estimates for new datasets in single fast forward passes, in a fraction of a second where likelihood-based inference could take hours. That survey spans point estimation, approximate Bayesian inference, learned summary statistics, and likelihood approximation; but, like the works above, it treats the variance of the training objective as incidental rather than as a quantity to be controlled. That variance is the gap this thesis takes up.

1.2.4 The simulation cost of likelihood-free inference

A separate strand of work attacks the efficiency of likelihood-free inference from the simulation side. Because these methods lean on enormous numbers of model simulations, much can be saved by spending the simulation budget wisely. Multifidelity and multilevel schemes combine cheap, approximate simulators with the expensive model of interest and allocate computation across fidelities to minimise cost at a fixed accuracy [29], an allocation that can be made adaptive, learning the relationship between fidelities on the fly to approach the optimal trade-off [22]. This

line is complementary to the present work: it reduces the number and cost of the simulations that feed inference, whereas this thesis reduces the variance of the training gradient extracted from a given set of simulations; the two could in principle be combined.

1.2.5 Variance reduction for stochastic gradients

The technique developed here belongs to the literature on variance reduction for stochastic optimisation, which includes control variates and related estimators [28]. Rao-Blackwellisation in particular has been used to reduce the variance of gradient estimators elsewhere in machine learning, notably for the straight-through Gumbel-softmax estimator [?] and for stochastic gradients of discrete distributions [?]. Those methods Rao-Blackwellise the gradient with respect to discrete latent variables or reparameterisation noise; the contribution of this thesis differs in what is conditioned on. The training gradient of a neural Bayes estimator is Rao-Blackwellised with respect to a conjugate block of the model parameters, exploiting the conditional conjugacy of the inference model itself: the same structure that makes Gibbs sampling possible.

1.3 Contributions

The contributions of this thesis are as follows.

1. **Frames the variance of the training gradient as an explicit, reducible target.** A NBE is trained by minimising an empirical (Monte Carlo) Bayes risk [26], so its training gradient is itself a Monte Carlo estimator with a variance: a property of how the loss is sampled, not of the inference problem. Recognising this brings the Monte Carlo variance-reduction toolbox to bear on the training of these estimators, which their literature has not previously addressed (Chapter 3).
2. **Derives the *Rao-Blackwellised loss*:** When a block of the parameter vector has a tractable conditional posterior, that block is integrated out of the loss analytically. For squared-error loss the construction reduces to substituting the conditional posterior mean for the sampled target, and the residual conditional variance contributes no gradient (Chapter 4).
3. **Proves that the resulting gradient-variance reduction is positive semi-definite** (Rao-Blackwellisation can never increase the variance of the training gradient, for any model or architecture), and shows that for a linear network the same reduction carries to the fitted network weights themselves (Chapter 4).
4. **Demonstrates the method empirically on Bayesian linear regression.** Across a grid of problem dimensions and batch sizes, the Rao-Blackwellised estimator attains a lower loss than the standard NBE in every configuration, closing about half of the gap to the theoretical minimum, the Bayes-optimal floor computed by Gibbs sampling, and matching the standard NBE at a quarter of the minibatch size (Chapter 4).
5. **Contrasts Rao-Blackwellisation with importance sampling,** a second variance-reduction technique, on an autoregressive model, where importance sampling returns a

negative result: establishing that its benefit is contingent on the problem, where the Rao-Blackwell guarantee is unconditional (Chapter 5).

1.4 Thesis outline

Chapter 2 reviews Markov chain Monte Carlo, the per-dataset incumbent against which amortised inference is contrasted, and collects three running case studies — the normal, linear-regression, and autoregressive models — used later as Bayes-optimal references for neural estimators. Chapter 3 develops the Bayes estimator framework, defines neural Bayes estimators, and establishes the central observation that their training uses Monte Carlo integration. Chapter 4 is the core of the thesis: it derives the Rao-Blackwellised loss, proves the gradient-variance reduction, analyses its effect on the converged loss, and tests the method on Bayesian linear regression. Chapter 5 develops importance sampling as a contrasting technique and applies it to an autoregressive model. Chapter 6 summarises the findings and discusses limitations and directions for further work.

Chapter 2

Bayesian Inference

Bayesian inference treats an unknown parameter vector $\boldsymbol{\theta} \in \Theta$ as a random quantity [9]. Given a likelihood $p(\mathbf{y} | \boldsymbol{\theta})$ for observed data $\mathbf{y}$ and a prior density $p(\boldsymbol{\theta})$, Bayes' rule combines the two into the posterior:

$$p(\boldsymbol{\theta} | \mathbf{y}) = \frac{p(\mathbf{y} | \boldsymbol{\theta}) p(\boldsymbol{\theta})}{p(\mathbf{y})}, \quad \text{where} \quad p(\mathbf{y}) = \int_{\Theta} p(\mathbf{y} | \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}. \quad (2.1)$$

For inference, we are quite often interested in summaries of the posterior distribution: posterior means, variances, quantiles, or more generally posterior expectations $\mathbb{E}[g(\boldsymbol{\theta}) | \mathbf{y}] = \int g(\boldsymbol{\theta}) p(\boldsymbol{\theta} | \mathbf{y}) d\boldsymbol{\theta}$ of some function g .

Two obstacles are commonly encountered in Bayesian inference. The first is that the marginal likelihood, $p(\mathbf{y})$, is difficult to evaluate. This means that we generally can only find the posterior up to a normalising constant,

$$p(\boldsymbol{\theta} | \mathbf{y}) \propto p(\mathbf{y} | \boldsymbol{\theta}) p(\boldsymbol{\theta}). \quad (2.2)$$

The second is that, even when $p(\mathbf{y})$ is available, $\boldsymbol{\theta}$ can be high-dimensional, making evaluation of posterior expectations computationally expensive.

Monte Carlo integration can be applied to help solve these problems, approximating an intractable expectation by a sample average. In the case of posterior inference, samples $\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(N)}$ are drawn from the posterior $p(\boldsymbol{\theta} | \mathbf{y})$, then

$$\mathbb{E}[g(\boldsymbol{\theta}) | \mathbf{y}] \approx \frac{1}{N} \sum_{i=1}^N g(\boldsymbol{\theta}^{(i)}), \quad (2.3)$$

with the approximation improving as N grows. The difficulty is the sampling step: the posterior (2.2) cannot, in general, be sampled directly. Markov chain Monte Carlo (MCMC) resolves this by producing the samples in (2.3) not independently but as the states of a Markov chain constructed to have the posterior as its limiting distribution.

2.1 Conjugate Models

Whether MCMC is needed depends entirely on the model. For a small class of *conjugate* models the posterior is itself a standard distribution that can be sampled directly; the draws in (2.3) are then genuinely independent, and plain Monte Carlo integration applies as written, with no chain required. Alternatively, conditional conjugacy may hold for individual blocks of parameters, where each block's conditional posterior is a standard distribution even when the joint posterior is not. This is the structure the Gibbs sampler (introduced in Section 2.2.2) exploits to build a chain. Additionally, for models with no such structure the posterior is known only up to proportionality (2.2); this is the general case that MCMC addresses by constructing posterior samples indirectly, as the states of a Markov chain.

2.1.1 A conjugate example: linear regression

The canonical conjugate model is Bayesian linear regression. The observations $\mathbf{y} \in \mathbb{R}^n$ are taken to be a combination of a linear function of a fixed design matrix $X \in \mathbb{R}^{n \times p}$ with noise,

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I),$$

Additionally, the priors on the coefficients $\boldsymbol{\beta} \in \mathbb{R}^p$ and the noise σ^2 can be chosen to be:

$$\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}, \sigma_0^2 I_p), \quad \tau \equiv \sigma^{-2} \sim \mathcal{U}[\tau_{\text{low}}, \tau_{\text{high}}], \quad (2.4)$$

where we introduce the noise precision τ for simplicity. The likelihood is then

$$p(\mathbf{y} | \boldsymbol{\beta}, \tau) = \frac{|\tau I|^{1/2}}{(2\pi)^{n/2}} \exp\left\{-\frac{\tau}{2}(\mathbf{y} - X\boldsymbol{\beta})^\top (\mathbf{y} - X\boldsymbol{\beta})\right\}$$

and the prior on the coefficients is

$$p(\boldsymbol{\beta}) = \frac{|\sigma_0^2 I_p|^{-1/2}}{(2\pi)^{p/2}} \exp\left\{-\frac{1}{2} \sigma_0^{-2} \boldsymbol{\beta}^\top \boldsymbol{\beta}\right\}.$$

Case 1: known τ . Now let us suppose that the noise variance σ^2 is known, and we are interested in the posterior over $\boldsymbol{\beta}$ alone. By applying Bayes' rule, the posterior is then

$$\begin{aligned} p(\boldsymbol{\beta} | \mathbf{y}, \tau) &\propto p(\mathbf{y} | \boldsymbol{\beta}, \tau) p(\boldsymbol{\beta}) \\ &\propto \exp\left\{-\frac{\tau}{2}(\mathbf{y} - X\boldsymbol{\beta})^\top (\mathbf{y} - X\boldsymbol{\beta}) - \frac{1}{2} \sigma_0^{-2} \boldsymbol{\beta}^\top \boldsymbol{\beta}\right\} \\ &\propto \exp\left\{-\frac{1}{2} [\tau (\mathbf{y} - X\boldsymbol{\beta})^\top (\mathbf{y} - X\boldsymbol{\beta}) + \sigma_0^{-2} \boldsymbol{\beta}^\top \boldsymbol{\beta}]\right\} \\ &\propto \exp\left\{-\frac{1}{2} [\boldsymbol{\beta}^\top (\sigma_0^{-2} I_p + \tau X^\top X) \boldsymbol{\beta} - 2 \tau \boldsymbol{\beta}^\top X^\top \mathbf{y}]\right\} \\ &\propto \exp\left\{-\frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\beta}_n)^\top \Sigma_n^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_n)\right\}, \end{aligned}$$

where we have defined the posterior covariance and mean as

$$\Sigma_n := (\sigma_0^{-2} I_p + \tau X^\top X)^{-1}, \quad \beta_n := \Sigma_n \tau X^\top \mathbf{y}. \quad (2.5)$$

Note that the posterior is Gaussian:

$$\beta | \mathbf{y}, \tau \sim \mathcal{N}(\beta_n, \Sigma_n). \quad (2.6)$$

We say that we have chosen a conjugate prior for β , because the posterior is in the same family as the prior.

Case 2: known β . Now let us suppose that the coefficients β are known, and we are interested in the posterior over the noise precision τ alone. By applying Bayes' rule, the posterior is then,

$$\begin{aligned} p(\tau | \mathbf{y}, \beta) &\propto p(\mathbf{y} | \beta, \tau) p(\tau) \\ &\propto \tau^{n/2} \exp\left\{-\frac{\tau}{2} (\mathbf{y} - X\beta)^\top (\mathbf{y} - X\beta)\right\} \mathbb{I}_{[\tau_{\text{low}} \leq \tau \leq \tau_{\text{high}}]}, \end{aligned}$$

where we have used the fact that the prior on τ is uniform, and therefore constant with respect to τ . The posterior is then a truncated Gamma distribution,

$$\tau | \mathbf{y}, \beta \sim \Gamma\left(\frac{n}{2} + 1, \frac{1}{2} (\mathbf{y} - X\beta)^\top (\mathbf{y} - X\beta)\right) \quad \text{on } (\tau_{\text{low}}, \tau_{\text{high}}). \quad (2.7)$$

Like Case 1, this is a conjugacy: a uniform prior is the flat, shape-one rate-zero member of the Gamma family, so the posterior is again a Gamma, here truncated to the prior support. What matters for the sampler of Section 2.2.2, though, is only that this full conditional is a standard distribution that can be drawn from directly, which it is.

In the above cases, the posterior is tractable and can be sampled directly. However, consider the model where both β and τ are unknown, and we are interested in the joint posterior over both parameters. We are able to still exploit these conditional conjugacies in MCMC, discussed next, in the Gibbs sampler. This property will also be fundamental in Chapter 4 on Rao-Blackwellisation.

2.2 Markov chain Monte Carlo

MCMC is a method to produce samples from a target distribution by constructing a Markov chain with the target as its equilibrium distribution [3, 11]. When the target is the posterior distribution of a Bayesian model, MCMC can be used even without appealing to conjugacy.

A Markov chain on Θ is specified by a transition kernel $T(\theta' | \theta)$, the density of moving to θ' from the current state θ . A distribution π is *stationary* for T if it is left invariant by the transition, that is, if $\int T(\theta' | \theta) \pi(\theta) d\theta = \pi(\theta')$. The design problem for MCMC is to construct a kernel whose stationary distribution is the target posterior. A convenient sufficient condition for stationarity is the following symmetry property.

Definition 2.1 (Detailed balance). A Markov chain with transition kernel $T(\boldsymbol{\theta}' | \boldsymbol{\theta})$ satisfies *detailed balance* with respect to a distribution π if

$$T(\boldsymbol{\theta}' | \boldsymbol{\theta}) \pi(\boldsymbol{\theta}) = T(\boldsymbol{\theta} | \boldsymbol{\theta}') \pi(\boldsymbol{\theta}') \quad \text{for all } \boldsymbol{\theta}, \boldsymbol{\theta}' \in \Theta. \quad (2.8)$$

A chain satisfying detailed balance with respect to π is called *reversible*, and π is then stationary for it. Integrating both sides of (2.8) over $\boldsymbol{\theta}$,

$$\int_{\Theta} T(\boldsymbol{\theta}' | \boldsymbol{\theta}) \pi(\boldsymbol{\theta}) d\boldsymbol{\theta} = \int_{\Theta} T(\boldsymbol{\theta} | \boldsymbol{\theta}') \pi(\boldsymbol{\theta}') d\boldsymbol{\theta} = \pi(\boldsymbol{\theta}') \int_{\Theta} T(\boldsymbol{\theta} | \boldsymbol{\theta}') d\boldsymbol{\theta} = \pi(\boldsymbol{\theta}'),$$

where the last equality holds because $T(\cdot | \boldsymbol{\theta}')$ is a probability density. Hence detailed balance implies stationarity.

Stationarity alone does not guarantee that a chain started from an arbitrary state converges to π . The further requirement is *ergodicity*: the chain must be irreducible (able to reach any region of Θ from any other) and aperiodic. A chain that is reversible with respect to π and ergodic converges to π from any starting state, and the sample average (2.3) converges to the corresponding expectation [3, 25]. This holds even though the states of the chain are not independent: successive draws are correlated, each generated from the one before.

The correlation between successive MCMC samples means that sample averages have higher variance than when using independent posterior draws. This can be quantified using effective sample size, which is the (typically smaller) number of independent samples that would give the same variance as MCMC sample averages.

In practice the early states, drawn before the chain has reached its stationary regime, are discarded as *burn-in*. Constructing an ergodic kernel that is reversible with respect to the posterior is the task of the algorithms that follow.

MCMC is not a single algorithm but a class of algorithms. Below, we describe several methods for constructing MCMC chains with the desired equilibrium distribution.

2.2.1 The Metropolis–Hastings algorithm

The Metropolis–Hastings algorithm [12, 17] builds a reversible chain from an arbitrary proposal mechanism by accepting or rejecting proposed moves. From the current state $\boldsymbol{\theta}$, a candidate $\boldsymbol{\theta}'$ is drawn from a *proposal distribution* $q(\boldsymbol{\theta}' | \boldsymbol{\theta})$ and accepted with probability

$$\alpha(\boldsymbol{\theta}, \boldsymbol{\theta}') = \min\left(1, \frac{p(\boldsymbol{\theta}' | \mathbf{y}) q(\boldsymbol{\theta} | \boldsymbol{\theta}')}{p(\boldsymbol{\theta} | \mathbf{y}) q(\boldsymbol{\theta}' | \boldsymbol{\theta})}\right); \quad (2.9)$$

otherwise the chain remains at $\boldsymbol{\theta}$. The procedure is given in Algorithm 1.

Algorithm 1: Metropolis–Hastings

Input: unnormalised posterior, proposal q , chain length N , initial state $\boldsymbol{\theta}^{(0)}$

Output: chain $\boldsymbol{\theta}^{(0)}, \dots, \boldsymbol{\theta}^{(N)}$

for $i = 0, 1, \dots, N - 1$ **do**

```
    draw  $\boldsymbol{\theta}' \sim q(\cdot | \boldsymbol{\theta}^{(i)})$ ;  
    compute  $\alpha(\boldsymbol{\theta}^{(i)}, \boldsymbol{\theta}')$  from (2.9);  
    draw  $u \sim \text{Uniform}(0, 1)$ ;  
    if  $u \leq \alpha(\boldsymbol{\theta}^{(i)}, \boldsymbol{\theta}')$  then  
         $\boldsymbol{\theta}^{(i+1)} \leftarrow \boldsymbol{\theta}'$ ;  
    else  
         $\boldsymbol{\theta}^{(i+1)} \leftarrow \boldsymbol{\theta}^{(i)}$ ;
```

The acceptance probability (2.9) depends on the posterior only through the ratio $p(\boldsymbol{\theta}' | \mathbf{y})/p(\boldsymbol{\theta} | \mathbf{y})$. By Bayes’ rule (2.1) the evidence $p(\mathbf{y})$ cancels from this ratio,

$$\frac{p(\boldsymbol{\theta}' | \mathbf{y})}{p(\boldsymbol{\theta} | \mathbf{y})} = \frac{p(\mathbf{y} | \boldsymbol{\theta}') p(\boldsymbol{\theta}')}{p(\mathbf{y} | \boldsymbol{\theta}) p(\boldsymbol{\theta})},$$

so the algorithm requires only the unnormalised posterior (2.2). This is what makes Metropolis–Hastings applicable to the generic intractable posterior. The chain it produces satisfies detailed balance with respect to the posterior, and is therefore reversible with the posterior as its stationary distribution [6]; the verification is a direct substitution of (2.9) into Definition 2.1, and is given in Appendix A.1. An application of the Metropolis–Hastings algorithm to estimate the parameters of a normal distribution is given in Example 1, introduced at the end of this chapter, and then used in Chapter 3 where it is used as a benchmark against a neural estimator.

This algorithm is general, but its efficiency and correctness depend sharply on the proposal q . A proposal that takes small steps yields a highly autocorrelated chain that explores Θ slowly; a proposal that takes large steps is frequently rejected and the chain stalls. Tuning q to balance these failure modes is difficult, and becomes more so as the dimension of $\boldsymbol{\theta}$ grows, particularly when the components of $\boldsymbol{\theta}$ differ in scale or are strongly correlated. For some special cases, we can avoid joint proposals entirely and instead update the components of $\boldsymbol{\theta}$ one block at a time. This is the idea behind the Gibbs sampler.

2.2.2 The Gibbs sampler

The Gibbs sampler [5, 10] can be applied whenever the *full conditional* distributions of the model are tractable. Partition the parameter vector into blocks $\boldsymbol{\theta} = (\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(B)})$ and write $\boldsymbol{\theta}^{(-b)}$ for all blocks but the b -th. The full conditional of block b is the distribution $p(\boldsymbol{\theta}^{(b)} | \boldsymbol{\theta}^{(-b)}, \mathbf{y})$ of that block given the data and the remaining blocks. The Gibbs sampler updates each block in turn by drawing it from its full conditional, as set out in Algorithm 2.

Algorithm 2: Gibbs sampler (systematic scan)

Input: full conditionals $p(\boldsymbol{\theta}^{(b)} | \boldsymbol{\theta}^{(-b)}, \mathbf{y})$, $b = 1, \dots, B$; chain length N ; initial state $\boldsymbol{\theta}^{(0)}$

Output: chain $\boldsymbol{\theta}^{(0)}, \dots, \boldsymbol{\theta}^{(N)}$

for $i = 0, 1, \dots, N - 1$ **do**

for $b = 1, \dots, B$ **do**
 draw $\boldsymbol{\theta}^{(i+1, b)} \sim p(\boldsymbol{\theta}^{(b)} | \boldsymbol{\theta}^{(i+1, 1)}, \dots, \boldsymbol{\theta}^{(i+1, b-1)}, \boldsymbol{\theta}^{(i, b+1)}, \dots, \boldsymbol{\theta}^{(i, B)}, \mathbf{y})$;

The Gibbs sampler involves many Metropolis–Hastings steps whose proposal for a given block b is the full conditional itself. Taking $q(\boldsymbol{\theta}^{(b)'} | \boldsymbol{\theta}) = p(\boldsymbol{\theta}^{(b)'} | \boldsymbol{\theta}^{(-b)}, \mathbf{y})$ in (2.9), and noting that the remaining blocks $\boldsymbol{\theta}^{(-b)}$ are held fixed,

$$\begin{aligned} \alpha &= \min \left(1, \frac{p(\boldsymbol{\theta}^{(b)'}, \boldsymbol{\theta}^{(-b)} | \mathbf{y}) p(\boldsymbol{\theta}^{(b)} | \boldsymbol{\theta}^{(-b)}, \mathbf{y})}{p(\boldsymbol{\theta}^{(b)}, \boldsymbol{\theta}^{(-b)} | \mathbf{y}) p(\boldsymbol{\theta}^{(b)'} | \boldsymbol{\theta}^{(-b)}, \mathbf{y})} \right) \\ &= \min \left(1, \frac{p(\boldsymbol{\theta}^{(b)'} | \boldsymbol{\theta}^{(-b)}, \mathbf{y}) p(\boldsymbol{\theta}^{(-b)} | \mathbf{y}) p(\boldsymbol{\theta}^{(b)} | \boldsymbol{\theta}^{(-b)}, \mathbf{y})}{p(\boldsymbol{\theta}^{(b)} | \boldsymbol{\theta}^{(-b)}, \mathbf{y}) p(\boldsymbol{\theta}^{(-b)} | \mathbf{y}) p(\boldsymbol{\theta}^{(b)'} | \boldsymbol{\theta}^{(-b)}, \mathbf{y})} \right) \\ &= 1 \end{aligned}$$

where the second line factorises the joint posterior as $p(\boldsymbol{\theta}^{(b)}, \boldsymbol{\theta}^{(-b)} | \mathbf{y}) = p(\boldsymbol{\theta}^{(b)} | \boldsymbol{\theta}^{(-b)}, \mathbf{y}) p(\boldsymbol{\theta}^{(-b)} | \mathbf{y})$, after which every factor cancels. The Gibbs sampler is therefore a Metropolis–Hastings algorithm with acceptance probability identically one: no proposal is ever rejected, and the tuning problem of Section 2.2.1 disappears.

The price is that the full conditionals must be available in closed form and easy to sample. This is most readily guaranteed when the model is *conditionally conjugate*: when, given the remaining blocks, each block has a prior conjugate to its likelihood, its full conditional belongs to a standard family. More generally, all the sampler needs is that each full conditional be a standard, samplable distribution. The recurring example in this thesis is the Bayesian linear regression of Section 2.1.1, whose two full conditionals were derived there: a Gaussian for the coefficients given the precision, a truncated Gamma for the precision given the coefficients. The Gibbs sampler alternates these two tractable draws, worked in full as Example 2, where Chapter 4 uses it to approximate a Bayes-optimal reference for the neural estimators of this thesis. The conditional structure that makes Gibbs sampling possible is, as Chapter 4 shows, exactly the structure that Rao-Blackwellisation exploits.

2.3 Case studies

The algorithms of this chapter are made concrete here on three models that recur throughout the thesis. Each is paired with the sampler suited to it — the normal model with Metropolis–Hastings, and the linear-regression and autoregressive models with the Gibbs sampler — and in every case the chain produces a posterior mean that a later chapter uses as the per-dataset reference against which a neural estimator is judged. Collecting the models here lets those chapters recall a model and its sampler in a line rather than re-derive them.

2.3.1 Example 1 — the normal model

Consider estimating the parameters of a normal model from n independent observations,

$$(y_1, \dots, y_n \mid \mu, \sigma) \stackrel{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2), \quad \mu \sim \mathcal{N}(\mu_0, \sigma_0^2), \quad \sigma \sim \text{Uniform}(0, A),$$

with estimand $\boldsymbol{\theta} = (\mu, \sigma)$; later experiments take $\mu_0 = 0$, $\sigma_0 = 1$ and $A = 5$. The model is a controlled test case rather than a genuine application: its posterior can be written in closed form up to a normalising constant, so an accurate reference can be computed for every test dataset and an amortised estimator checked against it.

The likelihood of the n observations is

$$p(\mathbf{y} \mid \mu, \sigma) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right),$$

and Bayes' theorem combines it with the priors to give the posterior, up to a normalising constant,

$$\begin{aligned} p(\mu, \sigma \mid \mathbf{y}) &\propto p(\mathbf{y} \mid \mu, \sigma) p(\mu) p(\sigma) \\ &\propto \left(\prod_{i=1}^n \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right) \right) \frac{1}{\sqrt{2\pi} \sigma_0} \exp\left(-\frac{(\mu - \mu_0)^2}{2\sigma_0^2}\right) \frac{1}{A} \\ &\propto \sigma^{-n} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2 - \frac{(\mu - \mu_0)^2}{2\sigma_0^2}\right), \end{aligned}$$

valid for $\sigma \in (0, A)$. Taking logarithms gives the log-posterior

$$\log p(\mu, \sigma \mid \mathbf{y}) = -n \log \sigma - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2 - \frac{(\mu - \mu_0)^2}{2\sigma_0^2} + C,$$

where C is the constant that comes out from this proportionality. Note that we also enforce it to be $-\infty$ when $\sigma \notin (0, A)$. The dependence on the data simplifies through the identity $\sum_{i=1}^n (y_i - \mu)^2 = \sum_{i=1}^n (y_i - \bar{y})^2 + n(\bar{y} - \mu)^2$, which gives

$$\log p(\mu, \sigma \mid \mathbf{y}) = -n \log \sigma - \frac{1}{2\sigma^2} (S + n(\bar{y} - \mu)^2) - \frac{(\mu - \mu_0)^2}{2\sigma_0^2} + C, \quad (2.10)$$

where $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ is the sample mean and $S = \sum_{i=1}^n (y_i - \bar{y})^2$ is $(n - 1)$ times the sample variance. The data enter the posterior only through the pair $(\bar{y}, S)$, the sufficient statistics of the normal model.

The joint posterior is not a standard distribution, but the log-posterior is all the Metropolis–Hastings algorithm requires. Its state is the parameter pair $\boldsymbol{\theta} = (\mu, \sigma)$. From the current state a candidate $\boldsymbol{\theta}' = (\mu', \sigma')$ is drawn from a typically symmetric random-walk proposal $q(\boldsymbol{\theta}' \mid \boldsymbol{\theta})$ (a Gaussian step centred on the current state) and accepted with the Metropolis–Hastings probability. Given a symmetric proposal there is equality in the proposal density for the forward

and reverse moves, $q(\boldsymbol{\theta}' | \boldsymbol{\theta}) = q(\boldsymbol{\theta} | \boldsymbol{\theta}')$ and it cancels from the acceptance ratio, leaving

$$\alpha(\boldsymbol{\theta}, \boldsymbol{\theta}') = \min\left(1, \exp\{\log p(\boldsymbol{\theta}' | \mathbf{y}) - \log p(\boldsymbol{\theta} | \mathbf{y})\}\right),$$

a difference of log-posteriors in which the unknown constant C cancels. A candidate with $\sigma' \notin (0, A)$ has log-posterior $-\infty$, so $\alpha = 0$ and it is rejected. Iterating this accept–reject step yields a chain whose states, after burn-in, are draws from the posterior; their average estimates the posterior mean $\mathbb{E}[\boldsymbol{\theta} | \mathbf{y}]$. Chapter 3 uses this Metropolis–Hastings reference to assess a neural estimator of the same model.

2.3.2 Example 2 — Bayesian linear regression

Recall the two conjugate full conditionals derived for this model in Section 2.1.1. Conditioned on the precision, the coefficients are Gaussian, $\boldsymbol{\beta} | \mathbf{y}, \tau \sim \mathcal{N}(\boldsymbol{\beta}_n, \Sigma_n)$, with covariance and mean $\Sigma_n = (\sigma_0^{-2} I_p + \tau X^\top X)^{-1}$ and $\boldsymbol{\beta}_n = \Sigma_n \tau X^\top \mathbf{y}$; conditioned on the coefficients, the precision is a truncated Gamma. The joint posterior $p(\boldsymbol{\beta}, \tau | \mathbf{y})$ combines the two and is not a standard distribution, but each conditional is, so a Gibbs sampler applies.

Starting from an initial precision $\tau^{(0)}$, sweep s updates the two blocks in turn, drawing each from its conditional evaluated at the other block’s current value:

$$\begin{aligned} \boldsymbol{\beta}^{(s)} | \mathbf{y}, \tau^{(s-1)} &\sim \mathcal{N}(\boldsymbol{\beta}_n(\tau^{(s-1)}), \Sigma_n(\tau^{(s-1)})), \\ \tau^{(s)} | \mathbf{y}, \boldsymbol{\beta}^{(s)} &\sim \text{Gamma}\left(\frac{n}{2} + 1, \frac{1}{2} \|\mathbf{y} - X\boldsymbol{\beta}^{(s)}\|^2\right) \text{ on } (\tau_{\text{low}}, \tau_{\text{high}}), \end{aligned}$$

with $\boldsymbol{\beta}_n$ and Σ_n recomputed at the updated precision. The coefficient draw is taken through the Cholesky factor of the precision matrix Σ_n^{-1} , so that Σ_n need never be formed explicitly; the precision draw samples the truncated Gamma by inverting its cumulative distribution function on $(\tau_{\text{low}}, \tau_{\text{high}})$.

After a burn-in of discarded sweeps, the retained draws $\{(\boldsymbol{\beta}^{(s)}, \tau^{(s)})\}_{s=1}^S$ are averaged — the noise scale of each recovered as $\sigma^{(s)} = (\tau^{(s)})^{-1/2}$ beforehand — to give the posterior mean,

$$\frac{1}{S} \sum_{s=1}^S (\boldsymbol{\beta}^{(s)}, \sigma^{(s)}) \longrightarrow \mathbb{E}[(\boldsymbol{\beta}, \sigma) | \mathbf{y}] \quad (S \rightarrow \infty).$$

Chapter 4 uses this sampler to compute a Bayes-optimal reference for its neural estimators, and the Gaussian $\boldsymbol{\beta}$ -conditional is also the structure its Rao-Blackwellised loss exploits.

2.3.3 Example 3 — the AR(1) model

The first-order autoregressive process

$$x_t = \rho x_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2), \quad (2.11)$$

has autoregressive coefficient $\rho \in (-1, 1)$ and noise scale $\sigma > 0$, with estimand $\boldsymbol{\theta} = (\rho, \sigma)$, priors $\rho \sim \text{Uniform}(-1, 1)$ and $\sigma \sim \text{Uniform}(0, A)$, and is initialised from its stationary distribution

$\mathcal{N}(0, \sigma^2/(1 - \rho^2))$. Like linear regression it is conditionally conjugate in both parameters, so a Gibbs sampler draws from the exact posterior; its two full conditionals are derived next, in parallel with the linear-regression conditionals above.

Conditioning on the first observation x_1 , the likelihood of the remaining $T - 1$ observations is

$$p(\mathbf{x} | \rho, \sigma^2) \propto (\sigma^2)^{-(T-1)/2} \exp\left(-\frac{1}{2\sigma^2} \sum_{t=2}^T (x_t - \rho x_{t-1})^2\right).$$

The autoregressive coefficient. As a function of ρ the exponent is quadratic. Writing $S_1 = \sum_{t=2}^T x_{t-1}^2$ and $S_{01} = \sum_{t=2}^T x_t x_{t-1}$,

$$\sum_{t=2}^T (x_t - \rho x_{t-1})^2 = \rho^2 S_1 - 2\rho S_{01} + \sum_{t=2}^T x_t^2,$$

so, with the uniform prior $p(\rho) = \frac{1}{2}$ on $(-1, 1)$,

$$p(\rho | \sigma^2, \mathbf{x}) \propto \exp\left(-\frac{1}{2\sigma^2} (\rho^2 S_1 - 2\rho S_{01})\right) \mathbf{1}\{|\rho| < 1\}.$$

Completing the square, $\rho^2 S_1 - 2\rho S_{01} = S_1 (\rho - S_{01}/S_1)^2 + \text{const}$, leaves a Gaussian kernel in ρ truncated by the prior:

$$\rho | \sigma^2, \mathbf{x} \sim \mathcal{N}\left(\frac{S_{01}}{S_1}, \frac{\sigma^2}{S_1}\right) \quad \text{on } (-1, 1).$$

The conditional mean S_{01}/S_1 is the ordinary least-squares estimate of ρ , and its conditional variance is σ^2/S_1 . Because $S_1 = \sum_t x_{t-1}^2$ grows as $|\rho| \rightarrow 1$ — the stationary variance $\sigma^2/(1 - \rho^2)$ inflates — the posterior for ρ sharpens towards the boundary, a fact Chapter 5 returns to.

The noise scale. Write the residual sum of squares as $\text{SSR} = \sum_{t=2}^T (x_t - \rho x_{t-1})^2$. The uniform prior $\sigma \sim \text{Uniform}(0, A)$ induces, by the change of variables $\sigma \mapsto \sigma^2$, the density $p(\sigma^2) \propto (\sigma^2)^{-1/2}$ on $(0, A^2)$. Hence

$$p(\sigma^2 | \rho, \mathbf{x}) \propto (\sigma^2)^{-(T-1)/2} \exp\left(-\frac{\text{SSR}}{2\sigma^2}\right) (\sigma^2)^{-1/2} = (\sigma^2)^{-T/2} \exp\left(-\frac{\text{SSR}}{2\sigma^2}\right),$$

the kernel of an inverse-Gamma density ($\text{Inv-Gamma}(a, b)$) is proportional to $(\sigma^2)^{-a-1} e^{-b/\sigma^2}$ with shape $a = \frac{T}{2} - 1$ and scale $b = \frac{1}{2}\text{SSR}$, truncated to the prior support:

$$\sigma^2 | \rho, \mathbf{x} \sim \text{Inv-Gamma}\left(\frac{T}{2} - 1, \frac{1}{2}\text{SSR}\right) \quad \text{on } (0, A^2).$$

The sampler. Each Gibbs sweep draws ρ then σ^2 from the two conditionals above. Both are standard distributions restricted to an interval, and both are sampled by the inverse-CDF method. For the truncated normal $\rho | \sigma^2, \mathbf{x}$, with conditional mean $m = S_{01}/S_1$ and standard deviation $s = \sigma/\sqrt{S_1}$, a draw

$$u \sim \text{Uniform}\left(\Phi\left(\frac{-1-m}{s}\right), \Phi\left(\frac{1-m}{s}\right)\right), \quad \rho = m + s \Phi^{-1}(u),$$

with Φ the standard normal cumulative distribution function, lands in $(-1, 1)$ by construction; the truncated inverse-Gamma $\sigma^2 | \rho, \mathbf{x}$ is sampled the same way through its own cumulative distribution function. After a burn-in of discarded sweeps, the retained draws are averaged to give the posterior mean $\mathbb{E}[(\rho, \sigma) | \mathbf{x}]$, the per-dataset reference used in Chapter 5.

2.4 The cost of per-dataset inference

Within their domains of applicability the algorithms of this chapter are reliable: each constructs an ergodic, reversible chain that converges to the exact posterior, and the approximation error is controlled by running the chain for longer. This reliability carries a structural cost. An MCMC chain targets the posterior $p(\boldsymbol{\theta} | \mathbf{y})$ for one specific dataset $\mathbf{y}$. When a new dataset is observed the entire procedure, warm-up, tuning, and the simulation of potentially many thousands of iterations, must be repeated, because the chain carries no transferable information from one inference problem to the next.

This is of little consequence when inference is required for a handful of datasets. It becomes prohibitive when the same model must be fitted to hundreds or thousands of distinct datasets: the total cost scales linearly in the number of inference problems, with no economy of scale. This limitation motivates *amortised* inference: rather than solving each problem independently, one invests computational effort once to learn a reusable mapping from data to posterior summaries, after which inference for any new dataset costs a single evaluation of that mapping. Constructing such a mapping with neural networks is the subject of Chapter 3.

Chapter 3

Amortisation and Neural Bayes Estimators

MCMC solves each inference problem in isolation: for every new dataset the chain must be run afresh, at a cost that is incurred each time. This chapter develops an alternative. Rather than approximating the full posterior for one dataset at a time, we target specific *posterior functionals* (summaries of the form $\mathbb{E}[g(\boldsymbol{\theta}) | \mathbf{y}]$) and we learn them with a single estimator map $\hat{\boldsymbol{\theta}}$, trained once, that returns the desired summary for any dataset drawn from the model. The idea is to exploit the *universal function-approximation* capability of neural networks to approximate the optimal mapping from data to these summaries.

The functionals we target are not arbitrary. Each is the solution of a decision problem: for a suitable loss function, the posterior summary $\mathbb{E}[g(\boldsymbol{\theta}) | \mathbf{y}]$ is the *Bayes estimator*, the estimator that is optimal with respect to the posterior distribution. Section 3.1 develops this decision theory and shows how the choice of loss selects which posterior summary the Bayes estimator returns: the posterior mean under squared-error loss, posterior quantiles under quantile loss.

The Bayes estimator is a fixed function of the data, but evaluating it directly is generally as difficult as the original inference problem. Section 3.2 introduces *amortised inference* (which approximates this function once, up front, and reuses the approximation across all subsequent datasets, so that the fitting cost is amortised over every later inference) and makes it concrete: the approximating function is a neural network which we call a *neural Bayes estimator*

$$\hat{\boldsymbol{\theta}}_{\gamma}(\mathbf{y}) \approx \mathbb{E}[g(\boldsymbol{\theta}) | \mathbf{y}],$$

trained on simulated data to minimise the empirical Bayes risk. We discuss how the network architecture should match the structure of the data (DeepSets for exchangeable observations, recurrent networks for sequential data, feed-forward networks otherwise) and close Section 3.4 with a worked example estimating the parameters of a normal model, where the amortised estimator is checked against an MCMC reference.

3.1 The Bayes estimator framework

Point estimation can be formalised through decision theory [1]. A point estimator is a function $\hat{\boldsymbol{\theta}} : \mathcal{Y} \rightarrow \Theta$ mapping observed data $\mathbf{y}$ to an estimate $\hat{\boldsymbol{\theta}}(\mathbf{y})$. A *loss function* $L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})$ is a function that quantifies the cost of reporting $\hat{\boldsymbol{\theta}}$ when the true parameter is $\boldsymbol{\theta}$.

The quality of an estimator at a fixed parameter value is its expected loss over data realisations, the *frequentist risk*

$$R(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}) = \mathbb{E}_{\mathbf{y}|\boldsymbol{\theta}}[L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}(\mathbf{y}))].$$

Averaging the frequentist risk over the prior gives the *Bayes risk*,

$$r(\hat{\boldsymbol{\theta}}) = \mathbb{E}_{\boldsymbol{\theta} \sim p(\boldsymbol{\theta})}[R(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})] = \mathbb{E}_{\boldsymbol{\theta}, \mathbf{y}}[L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}(\mathbf{y}))], \quad (3.1)$$

the second equality following because $p(\boldsymbol{\theta})p(\mathbf{y}|\boldsymbol{\theta})$ is the joint density of $(\boldsymbol{\theta}, \mathbf{y})$.

Definition 3.1 (Bayes estimator). The *Bayes estimator* is the estimator minimising the Bayes risk,

$$\hat{\boldsymbol{\theta}}^*(\cdot) = \arg \min_{\hat{\boldsymbol{\theta}}} r(\hat{\boldsymbol{\theta}}(\cdot)). \quad (3.2)$$

By the law of iterated expectation the Bayes risk (3.1) can be rewritten as $r(\hat{\boldsymbol{\theta}}) = \mathbb{E}_{\mathbf{y}}[\mathbb{E}_{\boldsymbol{\theta}|\mathbf{y}}[L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}(\mathbf{y}))]]$. Because the estimator is free to choose $\hat{\boldsymbol{\theta}}(\mathbf{y})$ separately for each $\mathbf{y}$, the Bayes risk is minimised by minimising the inner *posterior expected loss* $\mathbb{E}_{\boldsymbol{\theta}|\mathbf{y}}[L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})]$ pointwise. The Bayes estimator (3.2) is therefore

$$\hat{\boldsymbol{\theta}}^*(\mathbf{y}) = \arg \min_{\hat{\boldsymbol{\theta}}} \mathbb{E}_{\boldsymbol{\theta}|\mathbf{y}}[L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})]. \quad (3.3)$$

3.1.1 Loss functions and posterior summaries

A crucial feature of the Bayes estimator is that by choosing different loss functions, it can be made to target different summaries of the posterior. The two results below show the loss functions we need to target the posterior mean and posterior quantiles.

Proposition 3.2 (Squared-error loss). *Under the squared-error loss $L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}) = \|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|^2$, the Bayes estimator is the posterior mean $\hat{\boldsymbol{\theta}}^*(\mathbf{y}) = \mathbb{E}[\boldsymbol{\theta}|\mathbf{y}]$.*

Proof. Fix the data $\mathbf{y}$, so that $\hat{\boldsymbol{\theta}}$ is the constant vector to be chosen, and expand the squared norm inside the posterior expectation:

$$\begin{aligned} \mathbb{E}_{\boldsymbol{\theta}|\mathbf{y}}[\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|^2] &= \mathbb{E}_{\boldsymbol{\theta}|\mathbf{y}}[(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})] \\ &= \mathbb{E}_{\boldsymbol{\theta}|\mathbf{y}}[\boldsymbol{\theta}^\top \boldsymbol{\theta} - 2\hat{\boldsymbol{\theta}}^\top \boldsymbol{\theta} + \hat{\boldsymbol{\theta}}^\top \hat{\boldsymbol{\theta}}] \\ &= \mathbb{E}[\boldsymbol{\theta}^\top \boldsymbol{\theta} | \mathbf{y}] - 2\hat{\boldsymbol{\theta}}^\top \mathbb{E}[\boldsymbol{\theta} | \mathbf{y}] + \hat{\boldsymbol{\theta}}^\top \hat{\boldsymbol{\theta}}, \end{aligned}$$

where the last line uses linearity of expectation. Differentiating with respect to $\hat{\boldsymbol{\theta}}$ using the

gradient identities $\nabla_{\hat{\theta}}(\hat{\theta}^\top \mathbf{a}) = \mathbf{a}$ and $\nabla_{\hat{\theta}}(\hat{\theta}^\top \hat{\theta}) = 2\hat{\theta}$ gives

$$\nabla_{\hat{\theta}} \mathbb{E}_{\theta|\mathbf{y}} [\|\theta - \hat{\theta}\|^2] = -2\mathbb{E}[\theta|\mathbf{y}] + 2\hat{\theta}.$$

The objective is the sum of a constant, a term linear in $\hat{\theta}$, and the convex quadratic $\hat{\theta}^\top \hat{\theta}$; it is therefore convex in $\hat{\theta}$, and its unique minimiser is the stationary point. Setting the gradient to zero,

$$-2\mathbb{E}[\theta|\mathbf{y}] + 2\hat{\theta} = \mathbf{0} \quad \implies \quad \hat{\theta}^*(\mathbf{y}) = \mathbb{E}[\theta|\mathbf{y}].$$

Proposition 3.3 (Quantile loss). *For a scalar parameter θ , under the α -quantile loss $L_\alpha(\theta, \hat{\theta}) = (\alpha - \mathbf{1}_{\{\theta < \hat{\theta}\}})(\theta - \hat{\theta})$, the Bayes estimator is the α -quantile of the posterior.*

Proof. The α -quantile loss equals $(1 - \alpha)(\hat{\theta} - \theta)$ when $\theta < \hat{\theta}$ and $\alpha(\theta - \hat{\theta})$ when $\theta \geq \hat{\theta}$, so the posterior expected loss splits at $\theta = \hat{\theta}$ as

$$R(\hat{\theta}) = (1 - \alpha) \int_{-\infty}^{\hat{\theta}} (\hat{\theta} - \theta) p(\theta|\mathbf{y}) d\theta + \alpha \int_{\hat{\theta}}^{\infty} (\theta - \hat{\theta}) p(\theta|\mathbf{y}) d\theta.$$

Here $\hat{\theta}$ appears both in the limits of integration and in the integrands, so each term is differentiated by the Leibniz integral rule. In the first integral the boundary term carries the integrand evaluated at $\theta = \hat{\theta}$, namely $(\hat{\theta} - \hat{\theta}) = 0$, so only the integrand derivative survives:

$$\begin{aligned} \frac{d}{d\hat{\theta}} \int_{-\infty}^{\hat{\theta}} (\hat{\theta} - \theta) p(\theta|\mathbf{y}) d\theta &= \underbrace{(\hat{\theta} - \hat{\theta}) p(\hat{\theta}|\mathbf{y})}_{=0} + \int_{-\infty}^{\hat{\theta}} \frac{\partial}{\partial \hat{\theta}} [(\hat{\theta} - \theta) p(\theta|\mathbf{y})] d\theta \\ &= \int_{-\infty}^{\hat{\theta}} p(\theta|\mathbf{y}) d\theta \\ &= F_{\theta|\mathbf{y}}(\hat{\theta}), \end{aligned}$$

where $F_{\theta|\mathbf{y}}$ is the posterior cumulative distribution function. In the second integral the boundary term vanishes for the same reason, and differentiating the lower limit contributes a minus sign:

$$\begin{aligned} \frac{d}{d\hat{\theta}} \int_{\hat{\theta}}^{\infty} (\theta - \hat{\theta}) p(\theta|\mathbf{y}) d\theta &= \underbrace{-(\hat{\theta} - \hat{\theta}) p(\hat{\theta}|\mathbf{y})}_{=0} - \int_{\hat{\theta}}^{\infty} p(\theta|\mathbf{y}) d\theta \\ &= -(1 - F_{\theta|\mathbf{y}}(\hat{\theta})). \end{aligned}$$

Combining the two and simplifying,

$$\begin{aligned} R'(\hat{\theta}) &= (1 - \alpha) F_{\theta|\mathbf{y}}(\hat{\theta}) - \alpha(1 - F_{\theta|\mathbf{y}}(\hat{\theta})) \\ &= F_{\theta|\mathbf{y}}(\hat{\theta}) - \alpha. \end{aligned}$$

The posterior expected loss R is convex in $\hat{\theta}$, so its minimiser is its stationary point. Setting

$R'(\hat{\theta}) = 0$ gives

$$F_{\theta|\mathbf{y}}(\hat{\theta}^*) = \alpha,$$

which is precisely the definition of the α -quantile of the posterior.

The case $\alpha = \frac{1}{2}$ recovers the posterior median under absolute-error loss; symmetric pairs such as $\alpha \in \{0.025, 0.975\}$ give the endpoints of a central credible interval. This thesis works throughout with squared-error loss and hence with the posterior mean.

3.2 Neural Bayes estimators

The Bayes estimator (3.3) is a fixed function of the data: it returns the optimal estimate for every possible dataset, not just the one in hand. This is what makes it the right object to approximate: a single approximation, reused across datasets, which amortises the cost of fitting over every later inference, where MCMC must instead be re-run from scratch each time (Section 2.4). Approximating it requires first defining a *parametric family* of estimators in place of the intractable search over all functions, then fitting that family to a Monte Carlo estimate of an otherwise-intractable Bayes risk, and finally using a neural network as a flexible surrogate for this family. The three subsections below develop these in turn.

A parametric family of estimators. Definition 3.1 poses the Bayes estimator as the solution of a *variational* problem: the minimisation (3.2) ranges over all functions $\hat{\theta} : \mathcal{Y} \rightarrow \Theta$, an infinite-dimensional search that is no more tractable than the posterior it encodes. We make it tractable by restricting the search to a family of functions $\{\hat{\theta}_\gamma : \gamma \in \Gamma\}$ indexed by a finite-dimensional parameter γ . The Bayes risk is then a function of γ alone,

$$r(\gamma) = \mathbb{E}_{\theta, \mathbf{y}}[L(\theta, \hat{\theta}_\gamma(\mathbf{y}))], \quad (3.4)$$

and the variational problem of Definition 3.1 becomes an ordinary parametric optimisation,

$$\gamma^* = \arg \min_{\gamma \in \Gamma} r(\gamma), \quad (3.5)$$

over the finite-dimensional space Γ . Restricting to a finite-dimensional search means that the family may not contain the Bayes estimator exactly; the best it can do is the closest member, and the approximation is only as good as the family allows.

A Monte Carlo objective. The objective (3.4) is still an expectation over parameter–data pairs, taken with respect to the joint distribution $p(\theta)p(\mathbf{y} | \theta)$, and is no more available in closed form than the posterior. However, it can be approximated. The joint distribution is something we can sample from, even when the likelihood cannot be evaluated: draw a parameter from the prior, then simulate data from the model given that parameter. Drawing K such pairs gives the

Monte Carlo approximation:

$$r(\gamma) \approx \frac{1}{K} \sum_{k=1}^K L(\theta^{(k)}, \hat{\theta}_\gamma(\mathbf{y}^{(k)})), \quad (3.6)$$

the *empirical Bayes risk*, which replaces the intractable expectation (3.4) by an average we can actually compute. Minimising (3.6) over γ is a tractable surrogate for the parametric problem (3.5), and hence for the original problem. The construction is *likelihood-free*: it asks only that we can simulate from $p(\mathbf{y} | \theta)$, never that we evaluate the likelihood analytically.

A neural network surrogate. For the surrogate to be worthwhile the family $\{\hat{\theta}_\gamma\}$ must be flexible enough to contain a close approximation to the Bayes estimator, which may be a highly nonlinear function of the data. Neural networks are the natural choice: by their universal function-approximation property they can approximate the estimator map to arbitrary accuracy given sufficient capacity, while remaining a finite-dimensional family that (3.6) can be optimised over by gradient methods. Taking $\hat{\theta}_\gamma$ to be a neural network with weights γ , trained by minimising the empirical Bayes risk, yields a neural Bayes estimator [26].

The loss function chosen in (3.6) determines which posterior summary the trained network approximates, exactly as in Section 3.1.1. In summary, the procedure followed to obtain the trained network and apply it to an unseen dataset is as follows:

1. **Simulate.** Draw parameters $\theta^{(k)} \sim p(\theta)$ from the prior, then simulate n data samples $\mathbf{y}^{(k)} \sim p(\mathbf{y} | \theta^{(k)})$, with $\mathbf{y}^{(k)} \in \mathbb{R}^n$, for $k = 1, \dots, K$.
2. **Train.** Choose the weights $\hat{\gamma}$ to minimise the empirical Bayes risk (3.6).
3. **Apply.** For a new dataset $\mathbf{y}$, report $\hat{\theta}_{\hat{\gamma}}(\mathbf{y})$.

3.2.1 Training by stochastic gradient descent

Training minimises the empirical Bayes risk (3.6) by *stochastic gradient descent*. Gradient descent is an iterative method: starting from some initial weights, it repeatedly adjusts γ by a small step in the direction of steepest descent (the negative gradient $-\nabla_\gamma r(\gamma)$) so that the risk decreases from one step to the next, and continues until the risk levels off. Computing the exact gradient would mean summing over all K simulated pairs at every step, which is expensive. The stochastic variant avoids this: at each step it draws a small random *minibatch* of B pairs and uses the gradient of the loss on that minibatch in place of the full gradient. Written out, a minibatch $\{(\theta^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^B$ of pairs simulated from the model gives the average of the per-pair gradients,

$$\hat{g}_B(\gamma) = \frac{1}{B} \sum_{i=1}^B \nabla_\gamma L(\theta^{(i)}, \hat{\theta}_\gamma(\mathbf{y}^{(i)})).$$

Each pair is drawn from the joint distribution $p(\theta)p(\mathbf{y} | \theta)$, so interchanging differentiation and expectation shows this average to be unbiased for the gradient of the population risk (3.4),

$$\mathbb{E}[\hat{g}_B(\gamma)] = \nabla_\gamma \mathbb{E}_{\theta, \mathbf{y}}[L(\theta, \hat{\theta}_\gamma(\mathbf{y}))] = \nabla_\gamma r(\gamma).$$

The minibatch gradient is therefore an unbiased Monte Carlo estimate of the population gradient $\nabla_{\gamma} r(\gamma)$, the same quantity full-batch descent approximates: each step follows the true gradient on average, cheaply, but with sampling noise.

Like any Monte Carlo estimator, this training gradient has a variance. The variance is not a property of the inference problem but of how the loss is sampled, and it can be reduced by the standard Monte Carlo variance-reduction techniques, applied here to the training, as distinct from their application to the inference itself, which we investigate in Chapter 4 and Chapter 5.

3.3 Neural architectures

The construction of Section 3.2 leaves the family $\{\hat{\theta}_{\gamma}\}$ unspecified beyond the requirement that it be a neural network. In practice there are actually many different neural network architectures, with different desirable properties useful for solving varying statistical problems. A Bayes estimator inherits the symmetries of the model that generates the data: if the data are exchangeable the estimator is invariant to their ordering, if they are a time series the estimator depends on their order, and so on. Building these properties into the architecture, rather than hoping the network learns them, reduces the function class to one that already contains estimators of the right form; this makes training more efficient and stable. This section describes three architectures, each matched to a different data structure: DeepSets for independent and identically distributed (i.i.d.) observations, recurrent networks for sequential data, and feed-forward networks for data of fixed dimension with no such structure to exploit.

3.3.1 The DeepSets architecture

Many inference problems involve *replicated* data: a sample $\mathbf{y} = (y_1, \dots, y_n)$ of independent observations from a common model. A Bayes estimator for such data is invariant to the ordering of the observations, and the network should be too. The DeepSets architecture [30] enforces this permutation invariance by construction. Each observation is mapped through a shared network ψ to a high-dimensional embedding; the embeddings are combined by a permutation-invariant operation, typically summation; and a second network ϕ maps the aggregate to the estimate,

$$\hat{\theta}(\mathbf{y}) = \phi\left(\sum_{i=1}^n \psi(y_i)\right). \quad (3.7)$$

The sum $\sum_i \psi(y_i)$ plays the role of a learned summary statistic. For exponential-family models the sufficient statistic has exactly this additive form, and the architecture can be read as learning a sufficient statistic directly from simulated data, without the likelihood being known. Figure 3.1 summarises the data flow in this architecture.

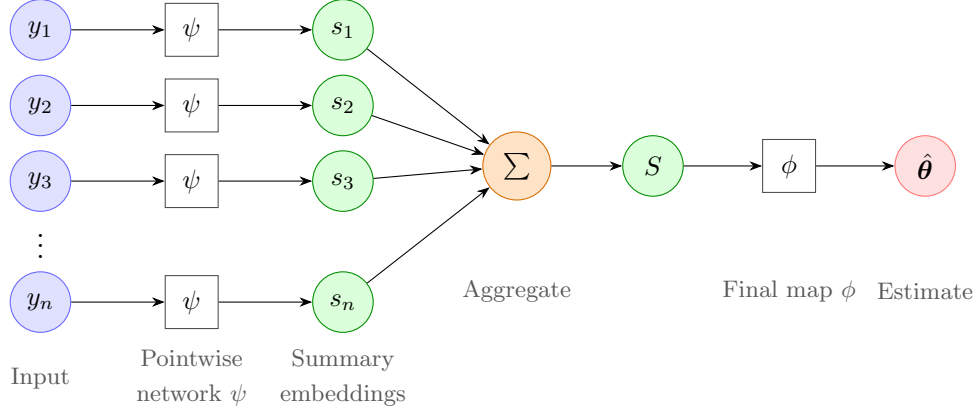


Figure 3.1: The DeepSets architecture for permutation-invariant estimation. Each observation y_i is mapped by the shared network ψ to an embedding s_i ; the embeddings are summed into the permutation-invariant aggregate $S = \sum_i \psi(y_i)$, which the second network ϕ maps to the estimate $\hat{\theta}(\mathbf{y})$.

3.3.2 Recurrent architectures for sequential data

DeepSets is appropriate only when the observations are exchangeable. For *sequential* data such as a time series $\mathbf{y} = (y_1, \dots, y_n)$ in which the ordering carries information, permutation invariance would discard precisely the dependence the parameters govern. The autoregressive model of Chapter 5 is the case in point: the AR(1) coefficient is identified by the correlation between successive observations, which a permutation-invariant summary cannot see.

The natural alternative is a *recurrent* network, which processes the observations in order while maintaining a hidden state $\mathbf{h}_t$ that summarises the sequence up to step t ,

$$\mathbf{h}_t = \psi(\mathbf{h}_{t-1}, y_t), \quad \hat{\theta}(\mathbf{y}) = \phi(\mathbf{h}_n).$$

Here ψ is a recurrent cell shared across steps and ϕ maps the terminal hidden state to the estimate; Figure 3.2 shows the network unrolled over the sequence. The hidden state plays the role that the sum $\sum_i \psi(y_i)$ plays in DeepSets, a learned summary of the data, but one accumulated in sequence and so sensitive to temporal order. The experiments of Chapter 5 take ψ to be a gated recurrent unit [7], a widely adopted recurrent block whose gating mitigates the vanishing gradients over long sequences.

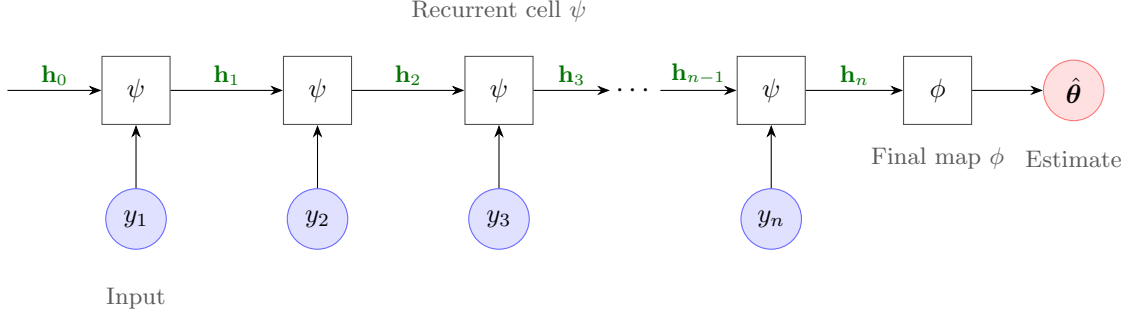


Figure 3.2: A recurrent architecture for sequential data, unrolled over the sequence. At step t the shared cell ψ combines the observation y_t with the previous hidden state $\mathbf{h}_{t-1}$ to produce $\mathbf{h}_t$; the final map ϕ turns the terminal hidden state $\mathbf{h}_n$ into the estimate $\hat{\theta}(\mathbf{y})$.

3.3.3 Feed-forward architectures for fixed-dimension data

When the data carry neither exchangeable nor sequential structure to exploit (as in linear regression with a fixed design matrix X , where each observation is tied to its own row of covariates and the observations are neither interchangeable nor ordered in time), there is no symmetry for the architecture to encode. The natural choice is then a *feed-forward* network, which takes the data as a fixed-length vector and maps it to the estimate through a sequence of affine transformations and nonlinearities,

$$\hat{\theta}(\mathbf{y}) = \phi(\mathbf{y}).$$

Unlike DeepSets and the recurrent network, the feed-forward architecture builds in no invariance and accommodates no change in the size of the data: the input dimension is fixed by the network, as suits a problem in which the number of observations and their meaning are fixed in advance. What it gives up in structure it makes up in generality, serving as the default when no exploitable structure is present.

3.4 Example 1: estimating normal parameters

As a concrete demonstration, we return to Example 1, the normal model: n independent observations $y_i \sim \mathcal{N}(\mu, \sigma^2)$ with priors $\mu \sim \mathcal{N}(0, 1)$ and $\sigma \sim \text{Uniform}(0, 5)$ and estimand $\theta = (\mu, \sigma)$. The model is a controlled test case rather than a genuine application: its posterior is available in closed form up to a normalising constant (the log-posterior (2.10)), so an accurate reference can be computed for every test dataset and the amortised estimator checked against it.

That reference is the Metropolis–Hastings sampler of Section 2.3.1. Run separately on each of 256 test datasets, its post-burn-in average estimates the posterior mean $\mathbb{E}[\theta | \mathbf{y}]$ — the same summary the squared-error-trained network targets (Proposition 3.2), so the two methods are compared on equal terms.

A neural Bayes estimator with the DeepSets architecture (3.7) was trained under squared-error loss on simulated data and compared against that reference. Figure 3.3 compares the two, and Table 3.1 summarises their accuracy and cost. They agree closely in accuracy: the mean squared

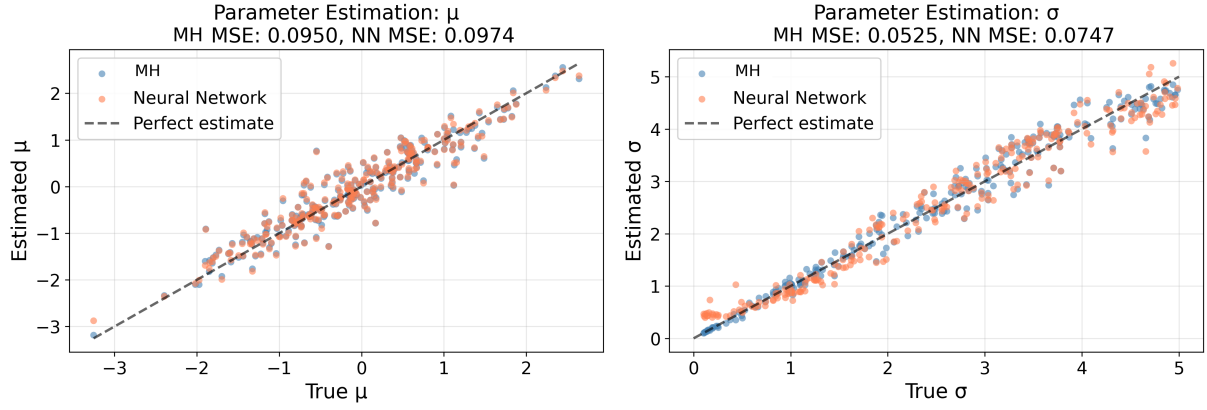


Figure 3.3: Estimated against true parameter values for the normal model, across 256 test datasets, for Metropolis–Hastings and the neural Bayes estimator. Both methods track the true parameters closely; the dashed line marks the hypothetical perfect estimation.

Table 3.1: Normal model: accuracy (mean squared error over the 256 test datasets) and per-dataset cost of the Metropolis–Hastings reference and the neural Bayes estimator. The two agree closely in accuracy, but the amortised network is faster per dataset by some three orders of magnitude (single-threaded CPU timing; 5,000-step chains for Metropolis–Hastings).

	μ (MSE)	σ (MSE)	Cost per dataset
Metropolis–Hastings	0.095	0.053	≈ 26 ms (re-run per dataset)
Neural Bayes estimator	0.097	0.075	$\approx 9 \mu$ s (one forward pass)

error of the neural estimator is within a few percent of that of Metropolis–Hastings for μ and somewhat larger for σ . They differ sharply in cost. Metropolis–Hastings must be run afresh for each dataset, whereas the trained network returns an estimate in a single forward pass — here roughly 2,800 times faster per dataset.

Chapter 4

Rao-Blackwellised Loss Functions

Chapter 3 established that training a neural Bayes estimator is Monte Carlo integration of the Bayes risk, so the training gradient is itself a Monte Carlo estimator with a variance. This chapter reduces that variance by *Rao-Blackwellisation*. Rather than estimating the Bayes risk by sampling the whole parameter vector, we integrate a conjugate block of it out of the loss analytically and Monte Carlo approximate only what remains. Under squared-error loss the construction is very simple: it replaces the sampled target by its conditional posterior mean, and the resulting training gradient is provably no noisier than the standard one, for any model and any architecture. For the special case when the network is linear, the guarantee strengthens further, from the gradient to the fitted weights themselves.

The chapter has three sections. Section 4.1 develops the theory, Rao-Blackwellising the loss function in general and then specialising to squared-error loss, where the construction reduces to substituting a conditional posterior mean and the training gradient it produces is provably no noisier than the standard one. Section 4.2 applies the method to Bayesian linear regression, the simplest model with a non-trivial conjugate block. Measured against a Gibbs-sampled Bayes-optimal reference, the Rao-Blackwellised estimator attains a lower error than the standard Monte Carlo estimator in every cell of a grid over parameter dimension and batch size, closing about half of the gap to the Bayes-optimal floor at the baseline, and matching the standard estimator at a quarter of the minibatch size. Section 4.4 discusses the result. This is the unconditional improvement the theory predicts, and the central empirical result of the thesis.

4.1 Rao-Blackwellising the loss function

4.1.1 The Rao-Blackwell theorem

The Rao-Blackwell theorem states that an estimator can always be improved, under convex loss, by conditioning it on a sufficient statistic [15].

Theorem 4.1 (Rao-Blackwell). *Let $\hat{\boldsymbol{\theta}}(\mathbf{y})$ be an estimator of $\boldsymbol{\theta}$ under a loss $L(\boldsymbol{\theta}, \cdot)$ convex in its second argument and with $\mathbb{E}_{\mathbf{y}|\boldsymbol{\theta}}[L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}(\mathbf{y}))] < \infty$, and let $T = T(\mathbf{y})$ be a statistic sufficient for*

θ . Define the conditioned estimator

$$\hat{\theta}_T(\mathbf{y}) = \mathbb{E}[\hat{\theta}(\mathbf{y}) | T]. \quad (4.1)$$

Then $\hat{\theta}_T$ is an estimator — it does not depend on the unknown θ — and its risk is no greater,

$$R(\theta, \hat{\theta}_T) \leq R(\theta, \hat{\theta}) \quad \text{for every } \theta. \quad (4.2)$$

Proof. Because T is sufficient for θ , the conditional distribution of $\mathbf{y}$ given T does not depend on θ ; hence neither does the conditional expectation defining $\hat{\theta}_T$ in (4.1), which is therefore a genuine estimator rather than a function of the parameter being estimated. Fix θ . Since $L(\theta, \cdot)$ is convex, the conditional form of Jensen's inequality gives, pointwise in T ,

$$L(\theta, \hat{\theta}_T(\mathbf{y})) = L(\theta, \mathbb{E}[\hat{\theta}(\mathbf{y}) | T]) \leq \mathbb{E}[L(\theta, \hat{\theta}(\mathbf{y})) | T].$$

Take the expectation over $\mathbf{y} | \theta$ on both sides; the tower property $\mathbb{E}[\mathbb{E}[\cdot | T]] = \mathbb{E}[\cdot]$ collapses the right-hand side, giving

$$R(\theta, \hat{\theta}_T) = \mathbb{E}[L(\theta, \hat{\theta}_T)] \leq \mathbb{E}[\mathbb{E}[L(\theta, \hat{\theta}) | T]] = \mathbb{E}[L(\theta, \hat{\theta})] = R(\theta, \hat{\theta}),$$

which is (4.2). When $L(\theta, \cdot)$ is strictly convex the inequality is strict unless $\hat{\theta}(\mathbf{y})$ is already a function of T alone, in which case conditioning leaves it unchanged.

The applications in this chapter use the theorem through the variance it controls, and the mechanism is the *law of total variance*: for any random vector Z and any conditioning variable T ,

$$\text{Var}(Z) = \mathbb{E}[\text{Var}(Z | T)] + \text{Var}(\mathbb{E}[Z | T]), \quad (4.3)$$

and since $\mathbb{E}[\text{Var}(Z | T)] \succeq 0$ in the positive semi-definite order,

$$\text{Var}(\mathbb{E}[Z | T]) \preceq \text{Var}(Z). \quad (4.4)$$

Replacing a random quantity by its conditional expectation cannot increase its variance, for any conditioning variable, not only a sufficient statistic, and strictly decreases it unless the quantity is already a function of T . Every variance reduction below is an instance of (4.4): in Section 4.1.4, with Z a Monte Carlo training gradient and $T = (\mathbf{y}, \theta_1)$; and in Section 4.1.5, with Z the fitted weights.

4.1.2 Rao-Blackwellising the Bayes risk

Let $\hat{\theta}_\gamma$ be a neural Bayes estimator with weights γ . By splitting the parameter vector into two blocks $\theta = (\theta_1, \theta_2)$, the Bayes risk can be written as:

$$\begin{aligned} r(\gamma) &= \mathbb{E}_{\theta, \mathbf{y}}[L(\theta, \hat{\theta}_\gamma(\mathbf{y}))] \\ &= \int_{\Theta_1} \int_{\mathcal{Y}} \int_{\Theta_2} L(\theta, \hat{\theta}_\gamma(\mathbf{y})) p(\theta_1, \mathbf{y}, \theta_2) d\theta_2 d\mathbf{y} d\theta_1. \end{aligned}$$

Factorising the joint density by the chain rule, $p(\boldsymbol{\theta}_1, \mathbf{y}, \boldsymbol{\theta}_2) = p(\boldsymbol{\theta}_2 | \mathbf{y}, \boldsymbol{\theta}_1) p(\mathbf{y} | \boldsymbol{\theta}_1) p(\boldsymbol{\theta}_1)$, the risk becomes a nested expectation,

$$r(\boldsymbol{\gamma}) = \mathbb{E}_{\boldsymbol{\theta}_1} \left[\mathbb{E}_{\mathbf{y} | \boldsymbol{\theta}_1} \left[\underbrace{\mathbb{E}_{\boldsymbol{\theta}_2 | \mathbf{y}, \boldsymbol{\theta}_1} [L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}))]}_{=: f(\boldsymbol{\theta}_1, \mathbf{y})} \right] \right]. \quad (4.5)$$

The inner expectation $f(\boldsymbol{\theta}_1, \mathbf{y})$ integrates $\boldsymbol{\theta}_2$ out against its conditional posterior $p(\boldsymbol{\theta}_2 | \mathbf{y}, \boldsymbol{\theta}_1)$. The premise of this chapter is that this inner integral is tractable, so that f can be evaluated in closed form, while the outer expectations over $\boldsymbol{\theta}_1$ and $\mathbf{y}$ cannot, and must be estimated by sampling.

The standard NBE training and the Rao-Blackwellised training differ in how much of (4.5) they leave to that sampling. Both draw the same K joint samples from the model, each comprising parameters $\boldsymbol{\theta}_1^{(k)}, \boldsymbol{\theta}_2^{(k)}$ and the data $\mathbf{y}^{(k)}$ they generate. The standard *Monte Carlo* objective estimates the whole nested expectation at once, feeding each sampled $\boldsymbol{\theta}_2^{(k)}$ into the loss; the *Rao-Blackwellised* objective uses $\boldsymbol{\theta}_2^{(k)}$ only to simulate the data $\mathbf{y}^{(k)}$, then discards it and evaluates the inner expectation f analytically,

$$\hat{r}_{\text{MC}}(\boldsymbol{\gamma}) = \frac{1}{K} \sum_{k=1}^K L(\boldsymbol{\theta}^{(k)}, \hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}^{(k)})), \quad (4.6)$$

$$\hat{r}_{\text{RB}}(\boldsymbol{\gamma}) = \frac{1}{K} \sum_{k=1}^K f(\boldsymbol{\theta}_1^{(k)}, \mathbf{y}^{(k)}), \quad (4.7)$$

where $f(\boldsymbol{\theta}_1, \mathbf{y}) = \mathbb{E}_{\boldsymbol{\theta}_2 | \mathbf{y}, \boldsymbol{\theta}_1} [L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}))]$ is the inner integral of (4.5), now evaluated in closed form. The standard objective (4.6) is exactly the empirical Bayes risk of Chapter 3; the Rao-Blackwellised objective (4.7) replaces each sampled loss term by its conditional expectation given $(\boldsymbol{\theta}_1^{(k)}, \mathbf{y}^{(k)})$, the draw $\boldsymbol{\theta}_2^{(k)}$ having served only to generate the data. Both are unbiased estimators of the same Bayes risk: taking the expectation of either (4.6) or (4.7) recovers (4.5). The two objectives therefore share the same minimiser, and differ only in variance.

When attempting to train the estimator by stochastic gradient descent, the variance of the gradients we are using to train the estimator becomes important. Recall that the population gradient is the gradient of the Bayes risk, and so will be unchanged,

$$g(\boldsymbol{\gamma}) = \nabla_{\boldsymbol{\gamma}} r(\boldsymbol{\gamma}) = \nabla_{\boldsymbol{\gamma}} \mathbb{E}_{\boldsymbol{\theta}, \mathbf{y}} [L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}))].$$

The empirical training gradients $\hat{g}_{\text{MC}}(\boldsymbol{\gamma}) = \nabla_{\boldsymbol{\gamma}} \hat{r}_{\text{MC}}(\boldsymbol{\gamma})$ and $\hat{g}_{\text{RB}}(\boldsymbol{\gamma}) = \nabla_{\boldsymbol{\gamma}} \hat{r}_{\text{RB}}(\boldsymbol{\gamma})$, however, will differ between the two schemes, since they are defined by the Monte Carlo estimates of the derivative of the Bayes risk with respect to the neural network weights $\boldsymbol{\gamma}$.

4.1.3 The squared-error case

Under squared-error loss $L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}) = \|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|^2$, we can go further, and show the inner integral $f(\mathbf{y}, \boldsymbol{\theta}_1)$ always simplifies, regardless of the model. If we write the estimator in block form $\hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}) = (\hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}^1(\mathbf{y}), \hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}^2(\mathbf{y}))$ to match the partition of $\boldsymbol{\theta}$, we can then separate the loss into compo-

nents where only the second term depends on θ_2 .

$$\begin{aligned}
f(\mathbf{y}, \theta_1) &= \mathbb{E}_{\theta_2 | \mathbf{y}, \theta_1} [\|\hat{\theta}_\gamma(\mathbf{y}) - \theta\|^2] \\
&= \mathbb{E}_{\theta_2 | \mathbf{y}, \theta_1} [\|\hat{\theta}_\gamma^1 - \theta_1\|^2 + \|\hat{\theta}_\gamma^2 - \theta_2\|^2] \\
&= \|\hat{\theta}_\gamma^1 - \theta_1\|^2 + \mathbb{E}_{\theta_2 | \mathbf{y}, \theta_1} [\|\hat{\theta}_\gamma^2 - \theta_2\|^2] \\
&= \|\hat{\theta}_\gamma^1 - \theta_1\|^2 + \|\hat{\theta}_\gamma^2 - \mathbb{E}[\theta_2 | \mathbf{y}, \theta_1]\|^2 + \text{tr}(\text{Var}(\theta_2 | \mathbf{y}, \theta_1)), \tag{4.8}
\end{aligned}$$

where the final line applies the identity $\mathbb{E}[\|Z - c\|^2] = \|\mathbb{E}[Z] - c\|^2 + \text{tr}(\text{Var}(Z))$ to the random vector θ_2 and the constant $\hat{\theta}_\gamma^2$.

Two features of (4.8) drive everything that follows. First, the sampled target θ_2 has been replaced by the *conditional posterior mean* $\mathbb{E}[\theta_2 | \mathbf{y}, \theta_1]$: Rao-Blackwellising a squared-error loss simply substitutes the conditional mean for the sampled value. Second, the trace term $\text{tr}(\text{Var}(\theta_2 | \mathbf{y}, \theta_1))$ does not depend on the weights γ . It contributes nothing to the gradient and need not be computed during training; it shifts the loss by a constant but leaves the optimisation unchanged.

In practice, what has been done is that the training pairs $(\mathbf{y}, \theta)$ have been replaced with pairs $(\mathbf{y}, \theta^{\text{RB}})$, where θ^{RB} has the same posterior mean but less variance than θ . Intuitively, this should improve training of the neural network. In the limit, if one had the posterior mean of θ already available, the network could be trained on $(\mathbf{y}, \mathbb{E}[\theta | \mathbf{y}])$ pairs. Of course, in this case, one would generally expect the neural network to be unnecessary except perhaps as a faster surrogate replacing direct analytical calculation of the posterior mean.

4.1.4 The training gradients

Training minimises the objective by stochastic gradient descent where each step moves the weights along an estimate of the gradient of the objective with respect to γ . The objects to compare between the two schemes are therefore not the objectives themselves but their gradients, and this section derives them.

Both follow from differentiating their respective squared-error loss,

$$\begin{aligned}
r_{\text{MC}}(\gamma) &= \mathbb{E}_\theta [\mathbb{E}_{\mathbf{y} | \theta} [\|\hat{\theta}_\gamma^1(\mathbf{y}) - \theta_1\|^2 + \|\hat{\theta}_\gamma^2(\mathbf{y}) - \theta_2\|^2]], \\
r_{\text{RB}}(\gamma) &= \mathbb{E}_{\theta_1} [\mathbb{E}_{\mathbf{y} | \theta_1} [\|\hat{\theta}_\gamma^1(\mathbf{y}) - \theta_1\|^2 + \|\hat{\theta}_\gamma^2(\mathbf{y}) - \mathbb{E}[\theta_2 | \mathbf{y}, \theta_1]\|^2 + \text{tr}(\text{Var}(\theta_2 | \mathbf{y}, \theta_1))]].
\end{aligned}$$

By differentiating each with respect to the weights γ , we obtain the gradients for the two schemes:

$$\begin{aligned}
g_{\text{MC}}(\gamma) &= 2 \mathbb{E}_{\theta, \mathbf{y}} [(\nabla_\gamma \hat{\theta}_\gamma^1(\mathbf{y}))^\top (\hat{\theta}_\gamma^1(\mathbf{y}) - \theta_1)] + 2 \mathbb{E}_{\theta, \mathbf{y}} [(\nabla_\gamma \hat{\theta}_\gamma^2(\mathbf{y}))^\top (\hat{\theta}_\gamma^2(\mathbf{y}) - \theta_2)], \\
g_{\text{RB}}(\gamma) &= 2 \mathbb{E}_{\theta_1, \mathbf{y}} [(\nabla_\gamma \hat{\theta}_\gamma^1(\mathbf{y}))^\top (\hat{\theta}_\gamma^1(\mathbf{y}) - \theta_1)] + 2 \mathbb{E}_{\theta_1, \mathbf{y}} [(\nabla_\gamma \hat{\theta}_\gamma^2(\mathbf{y}))^\top (\hat{\theta}_\gamma^2(\mathbf{y}) - \mathbb{E}[\theta_2 | \mathbf{y}, \theta_1])].
\end{aligned}$$

To show that $g_{\text{RB}}(\gamma)$ is less noisy than $g_{\text{MC}}(\gamma)$, we examine the variance of the stochastic

components inside the expectations. Specifically, we look at the second term of each gradient:

$$\begin{aligned} G_{\text{MC}}^{(2)} &= 2 J_{\gamma}(\mathbf{y})^{\top} (\hat{\boldsymbol{\theta}}_{\gamma}^2(\mathbf{y}) - \boldsymbol{\theta}_2), \\ G_{\text{RB}}^{(2)} &= 2 J_{\gamma}(\mathbf{y})^{\top} (\hat{\boldsymbol{\theta}}_{\gamma}^2(\mathbf{y}) - \mathbb{E}[\boldsymbol{\theta}_2 | \mathbf{y}, \boldsymbol{\theta}_1]), \quad \text{where} \quad J_{\gamma}(\mathbf{y}) \equiv \nabla_{\gamma} \hat{\boldsymbol{\theta}}_{\gamma}^2(\mathbf{y}). \end{aligned}$$

Note here that $G_{\text{RB}}^{(2)} = \mathbb{E}[G_{\text{MC}}^{(2)} | \mathbf{y}, \boldsymbol{\theta}_1]$: conditional on $(\mathbf{y}, \boldsymbol{\theta}_1)$ the estimate $\hat{\boldsymbol{\theta}}_{\gamma}^2(\mathbf{y})$ and its Jacobian are fixed, and the sampled $\boldsymbol{\theta}_2$ enters $G_{\text{MC}}^{(2)}$ linearly, so conditioning simply replaces $\boldsymbol{\theta}_2$ by its conditional mean. The Rao-Blackwellised gradient term is thus the conditional expectation of the Monte Carlo one, and by the tower property the two share the same mean, $\mathbb{E}[G_{\text{RB}}^{(2)}] = \mathbb{E}[G_{\text{MC}}^{(2)}]$, so the substitution introduces no bias; and the law of total variance (4.3) gives a gap that is an average of conditional covariances,

$$\text{Var}(G_{\text{MC}}^{(2)}) - \text{Var}(G_{\text{RB}}^{(2)}) = \mathbb{E}[\text{Var}(G_{\text{MC}}^{(2)} | \mathbf{y}, \boldsymbol{\theta}_1)] \succeq 0. \quad (4.9)$$

This gap concerns only the second term. The first is unaffected:

$$G^{(1)} = 2 (\nabla_{\gamma} \hat{\boldsymbol{\theta}}_{\gamma}^1(\mathbf{y}))^{\top} (\hat{\boldsymbol{\theta}}_{\gamma}^1(\mathbf{y}) - \boldsymbol{\theta}_1)$$

is already a function of $(\mathbf{y}, \boldsymbol{\theta}_1)$, so $\mathbb{E}[G^{(1)} | \mathbf{y}, \boldsymbol{\theta}_1] = G^{(1)}$. Combined with $G_{\text{RB}}^{(2)} = \mathbb{E}[G_{\text{MC}}^{(2)} | \mathbf{y}, \boldsymbol{\theta}_1]$, the entire per-sample gradient is the conditional expectation of the standard one, and averaging over the minibatch preserves this, so $\hat{g}_{\text{RB}} = \mathbb{E}[\hat{g}_{\text{MC}} | \mathbf{y}, \boldsymbol{\theta}_1]$. The law of total variance (4.3) therefore applies to the whole gradient vector, not merely its second term:

$$\text{Var}(\hat{g}_{\text{MC}}) - \text{Var}(\hat{g}_{\text{RB}}) = \mathbb{E}[\text{Var}(\hat{g}_{\text{MC}} | \mathbf{y}, \boldsymbol{\theta}_1)] \succeq 0.$$

This is positive semi-definite. Rao-Blackwellisation can therefore never increase the variance of the training gradient, and, since nothing in the argument used any property of the network, the guarantee holds for every model and every architecture.

To quantify this *variance gap*, we can substitute $G_{\text{MC}}^{(2)}$ into (4.9) and apply the identity $\text{Var}(AZ) = A \text{Var}(Z) A^{\top}$ for random vector Z and constant matrix A .

$$\text{Var}(G_{\text{MC}}^{(2)} | \mathbf{y}, \boldsymbol{\theta}_1) = 4 J_{\gamma}^{\top} \text{Var}(\boldsymbol{\theta}_2 | \mathbf{y}, \boldsymbol{\theta}_1) J_{\gamma},$$

This shows the reduction scales with two interpretable quantities. The first is the conditional posterior covariance $\text{Var}(\boldsymbol{\theta}_2 | \mathbf{y}, \boldsymbol{\theta}_1)$, which is large exactly when $\boldsymbol{\theta}_2$ is weakly identified by the data — the case where the sampled target is noisiest and stands to gain the most from being replaced. The second is the Jacobian J_{γ} , which is generally largest early in training, when the network weights are far from optimal. Taken together, these suggest that the variance reduction is greatest precisely when the optimiser most needs a clean signal: early in training, and on weakly identified parameters.

4.1.5 The linear network case

Rao-Blackwellisation reduces the variance of the training gradient; it is natural to ask what this implies for the estimator that training produces. For a *linear network*, one whose output is linear in its weights, the question can be answered exactly.

A linear network passes the data through a fixed feature map $z(\mathbf{y})$ and reads the estimate off as a linear combination of the resulting features,

$$\hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}) = z(\mathbf{y})^\top \boldsymbol{\gamma}, \quad (4.10)$$

with weight vector $\boldsymbol{\gamma}$. Because the feature map is fixed, the estimate is linear in $\boldsymbol{\gamma}$.

Consider K simulated datasets, dataset k carrying the estimand $\boldsymbol{\theta}^{(k)}$ and data $\mathbf{y}^{(k)}$. Collect the estimands into the rows of a matrix Θ and the feature vectors $z(\mathbf{y}^{(k)})$ into the rows of a matrix Z . Fitting the linear network (4.10) by least squares over the K datasets gives the weight vector

$$\hat{\boldsymbol{\gamma}} = (Z^\top Z)^{-1} Z^\top \Theta. \quad (4.11)$$

Both training schemes fit on the same K datasets, so they share the feature matrix Z and hence the matrix $A := (Z^\top Z)^{-1} Z^\top$ of (4.11); the schemes differ only in the estimand matrix Θ . Standard training stacks the sampled estimands into Θ_{MC} ; Rao-Blackwellised training stacks the same rows with each $\boldsymbol{\theta}_2$ -block replaced by its conditional posterior mean. Writing $T = \{(\mathbf{y}^{(k)}, \boldsymbol{\theta}_1^{(k)})\}_{k=1}^K$ for the data and the $\boldsymbol{\theta}_1$ -blocks of every dataset, and using the fact the datasets are simulated independently, the two estimand matrices are related, row by row, by a single conditional expectation,

$$\Theta_{\text{RB}} = \mathbb{E}[\Theta_{\text{MC}} | T]. \quad (4.12)$$

Column j of the weight matrix is $\hat{\boldsymbol{\gamma}}_{(j)} = A \Theta_{(j)}$. Because A is built from the data alone it is fixed given T ; combining this with (4.12), the constant matrix passes straight through the conditional expectation,

$$\hat{\boldsymbol{\gamma}}_{(j)}^{\text{RB}} = A \Theta_{(j)}^{\text{RB}} = A \mathbb{E}[\Theta_{(j)}^{\text{MC}} | T] = \mathbb{E}[A \Theta_{(j)}^{\text{MC}} | T] = \mathbb{E}[\hat{\boldsymbol{\gamma}}_{(j)}^{\text{MC}} | T].$$

The fitted Rao-Blackwellised weights are therefore the Rao-Blackwellisation of the fitted standard weights, with T the conditioning variable. The law of total variance (4.4) applies again,

$$\text{Var}(\hat{\boldsymbol{\gamma}}_{(j)}^{\text{MC}}) - \text{Var}(\hat{\boldsymbol{\gamma}}_{(j)}^{\text{RB}}) = \mathbb{E}[\text{Var}(\hat{\boldsymbol{\gamma}}_{(j)}^{\text{MC}} | T)] \succeq 0. \quad (4.13)$$

For a linear network, then, Rao-Blackwellisation not only reduces the variance of the training gradient; it reduces the variance of the fitted weights themselves, for any finite training set. The Rao-Blackwellised estimator makes more efficient use of the same simulated data.

A fully nonlinear network does not admit the closed form (4.11), and no such exact statement is available in general. However, the linear result is suggestive of the nonlinear case: if every layer but the last is held fixed, training the final layer is exactly the linear problem above, with z the feature map computed by the fixed layers, so the argument (4.10)–(4.13) applies to the final layer (with linear activation) of any network, be it linear or nonlinear. A complete treatment of

the nonlinear case is left to future work.

4.2 Application to Bayesian linear regression

4.2.1 The Bayesian linear regression model

We tested the method on Bayesian linear regression, Example 2 from Section 2.3.2. The model (2.4) is $\beta \sim \mathcal{N}(\mathbf{0}, \sigma_0^2 I_p)$, $\tau \sim \text{Uniform}(\tau_{\text{low}}, \tau_{\text{high}})$, and $\mathbf{y} | \beta, \tau \sim \mathcal{N}(X\beta, \tau^{-1} I_n)$ for a fixed design matrix $X \in \mathbb{R}^{n \times p}$. This model is an ideal test case for Rao-Blackwellisation because it is simple and well-understood. The estimand is $\theta = (\beta^\top, \sigma)^\top$, of dimension $p + 1$. In the notation of Section 4.1, $\theta_2 = \beta$ is the conjugate block to be integrated out and $\theta_1 = \sigma$ is the sampled block.

The model is conditionally conjugate in both blocks, as Section 2.1.1 derives. Two of those conditionals matter here. Given the precision, the coefficients have the Gaussian conditional $\beta | \mathbf{y}, \tau \sim \mathcal{N}(\beta_n, \Sigma_n)$ of (2.6), with covariance and mean $\Sigma_n = (\sigma_0^{-2} I_p + \tau X^\top X)^{-1}$ and $\beta_n = \Sigma_n \tau X^\top \mathbf{y}$ of (2.5); given the coefficients, the precision has the truncated-Gamma conditional (2.7). The Gaussian β -conditional is the structure that the Rao-Blackwellised loss of Section 4.2.2 exploits, and both conditionals drive the Gibbs sampler used for the Bayes floor in Section 4.2.4.

4.2.2 Monte Carlo and Rao-Blackwellised training losses

The standard (Monte Carlo) per-sample loss is the squared error against the sampled parameters,

$$\ell_{\text{MC}}(\gamma) = \|\hat{\theta}_\gamma^\beta(\mathbf{y}) - \beta\|^2 + (\hat{\theta}_\gamma^\sigma(\mathbf{y}) - \sigma)^2.$$

Applying the squared-error construction of Section 4.1.3 with the conjugate posterior (2.6) gives the Rao-Blackwellised loss

$$\ell_{\text{RB}}(\gamma) = \|\hat{\theta}_\gamma^\beta(\mathbf{y}) - \beta_n(\sigma, \mathbf{y})\|^2 + \text{tr}(\Sigma_n) + (\hat{\theta}_\gamma^\sigma(\mathbf{y}) - \sigma)^2, \quad (4.14)$$

in which the sampled β is replaced by the conditional mean β_n and the trace term $\text{tr}(\Sigma_n)$, being independent of γ , contributes no gradient. The two losses have the same population minimiser; they differ only in the variance of the stochastic gradient, exactly as Section 4.1 describes.

4.2.3 Quantifying the variance reduction

The trace term in (4.14) is constant in the network weights, so it does not enter the optimisation. It is nevertheless the quantity that explains when Rao-Blackwellisation should be most effective. Conditional on $(\mathbf{y}, \tau)$, the sampled coefficient target decomposes as

$$\beta = \beta_n + \varepsilon, \quad \varepsilon | \mathbf{y}, \tau \sim \mathcal{N}(\mathbf{0}, \Sigma_n).$$

Thus the Monte Carlo target differs from the Rao-Blackwellised target by a zero-mean posterior fluctuation with covariance Σ_n . For any beta-head prediction $\hat{\theta}_\gamma^\beta(\mathbf{y})$, the conditional squared-

error risk satisfies

$$\mathbb{E} \left[\|\hat{\boldsymbol{\theta}}_{\gamma}^{\beta}(\mathbf{y}) - \boldsymbol{\beta}\|^2 \mid \mathbf{y}, \tau \right] = \|\hat{\boldsymbol{\theta}}_{\gamma}^{\beta}(\mathbf{y}) - \boldsymbol{\beta}_n\|^2 + \text{tr}(\Sigma_n).$$

The term $\text{tr}(\Sigma_n)$ is therefore the conditional target variance seen by the Monte Carlo loss and removed by the Rao-Blackwellised loss.

The same quantity controls the gradient noise. Let $J_{\gamma}(\mathbf{y}) = \nabla_{\gamma} \hat{\boldsymbol{\theta}}_{\gamma}^{\beta}(\mathbf{y})$ denote the Jacobian of the beta-head with respect to the network weights. The beta part of the Monte Carlo gradient is

$$G_{\text{MC}}^{\beta} = 2J_{\gamma}(\mathbf{y})^{\top} \left(\hat{\boldsymbol{\theta}}_{\gamma}^{\beta}(\mathbf{y}) - \boldsymbol{\beta} \right),$$

while the Rao-Blackwellised gradient replaces $\boldsymbol{\beta}$ by $\boldsymbol{\beta}_n$. Conditional on $(\mathbf{y}, \tau)$, the latter is fixed and the only remaining randomness in G_{MC}^{β} is the posterior fluctuation $\boldsymbol{\beta} - \boldsymbol{\beta}_n$. Hence

$$\text{Var} \left(G_{\text{MC}}^{\beta} \mid \mathbf{y}, \tau \right) = 4J_{\gamma}(\mathbf{y})^{\top} \Sigma_n J_{\gamma}(\mathbf{y}),$$

which is precisely the conditional gradient variance removed by Rao-Blackwellisation.

The size of this removed component can be read directly from the spectrum of the design matrix. If $\lambda_1, \dots, \lambda_p$ are the eigenvalues of $X^{\top}X$, then

$$\text{tr}(\Sigma_n) = \sum_{j=1}^p \frac{1}{\sigma_0^{-2} + \tau \lambda_j} = \sigma_0^2 \sum_{j=1}^p \frac{1}{1 + \tau \sigma_0^2 \lambda_j}. \quad (4.15)$$

Each summand corresponds to one coefficient direction. Directions with large λ_j are well identified by the data and contribute little posterior variance; directions with small λ_j remain uncertain and contribute close to the prior variance σ_0^2 . Thus Rao-Blackwellisation should be most effective in regimes where many coefficient directions are weakly identified: high-dimensional problems, ill-conditioned designs, noisy observations, or small sample sizes.

Under the uniform precision prior used in the experiment, $\tau \sim \text{Uniform}(\tau_{\text{low}}, \tau_{\text{high}})$, the expected removed target variance has the closed form

$$\mathbb{E}_{\tau}[\text{tr}(\Sigma_n)] = \frac{1}{\tau_{\text{high}} - \tau_{\text{low}}} \sum_{j=1}^p \frac{\log(\sigma_0^{-2} + \tau_{\text{high}} \lambda_j) - \log(\sigma_0^{-2} + \tau_{\text{low}} \lambda_j)}{\lambda_j},$$

4.2.4 The Gibbs benchmark

To provide us with a reference for both neural networks' training performance, the natural yardstick is the lowest loss that any estimator can attain, the *Bayes-optimal floor*,

$$\begin{aligned} L^* &= \mathbb{E}_{\mathbf{y}} \left[\mathbb{E}_{\boldsymbol{\theta} \mid \mathbf{y}} [\|\boldsymbol{\theta} - \mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]\|^2] \right] \\ &= \mathbb{E}_{\mathbf{y}} [\text{tr Var}(\boldsymbol{\theta} \mid \mathbf{y})], \end{aligned} \quad (4.16)$$

attained by the posterior mean, so the floor is the expected posterior variance: the irreducible uncertainty that remains once the data have been seen. A network whose loss reaches L^* is

doing as well as any estimator could.

The floor cannot be evaluated directly: neither the posterior mean nor the posterior variance in (4.16) is available in closed form, because the joint posterior $p(\boldsymbol{\beta}, \tau | \mathbf{y})$ is not a standard distribution. Its two conditionals are, however, standard (Section 2.1.1), so it can be sampled by the Gibbs sampler of Example 2 (Section 2.3.2): each sweep draws $\boldsymbol{\beta}$ from the Gaussian conditional (2.6) through the Cholesky factor of Σ_n^{-1} , and τ from the truncated-Gamma conditional (2.7) by inverse-CDF sampling.

Gibbs as a benchmark. The sampler is run on a held-out test set of problems. After a burn-in of discarded sweeps, the retained draws $\{(\boldsymbol{\beta}^{(s)}, \tau^{(s)})\}_{s=1}^S$ are post-processed: the noise scale of each draw is recovered as $\sigma^{(s)} = (\tau^{(s)})^{-1/2}$ before averaging. The chain average then converges to the posterior mean as $S \rightarrow \infty$.

$$\frac{1}{S} \sum_s (\boldsymbol{\beta}^{(s)}, \sigma^{(s)}) \longrightarrow \mathbb{E}[\boldsymbol{\theta} | \mathbf{y}]$$

With the posterior mean in hand for each test problem, the benchmark L^* is estimated as the mean squared error of that estimate against the true $\boldsymbol{\theta}$, averaged over the test set. It is therefore computed in exactly the way the test loss of the neural estimators is, so the comparison of Section 4.2.6 is like-for-like.

4.2.5 Experimental design

The estimator is a three-layer multilayer perceptron of hidden width 128 with ReLU activations and a softplus output on the σ -head to enforce positivity, trained with Adam [14] at learning rate 10^{-3} . The two schemes are trained from identical initialisations on the same simulated batches, so that any difference between them is attributable to the loss alone. The experiment sweeps the parameter dimension $p \in \{2, 5, 10, 20, 30\}$ and the minibatch size $\{4, 16, 64, 256\}$ at a fixed sample size $n = 50$, training each cell for 200 epochs over ten seeds. The precision prior is $\tau = \sigma^{-2} \sim \text{Uniform}(0.01, 1)$, placing the noise scale in $\sigma \in (1, 10)$.

A single design matrix X is fixed per dimension. Because the population loss depends on X , each panel of Figure 4.1 corresponds to one inference task and results are aggregated only over training seeds, never pooled across design matrices. The Bayes-optimal floor L^* for each dimension is computed by the Gibbs sampler of Section 4.2.4 on a held-out test set, in exactly the way the test loss of the neural estimators is, so the comparison of Section 4.2.6 is like-for-like.

4.2.6 Results

Rao-Blackwellisation beats Monte Carlo in every cell. Figure 4.1 shows the training curves of the two estimators across the grid of parameter dimension p and batch size, at the baseline sample size $n = 50$, as the seed mean with a ± 1 standard-deviation band. The Rao-Blackwellised runs sit below the Monte Carlo runs in every panel, and the separation persists across all four batch sizes: a larger batch lowers both methods but does not close the gap between

them.

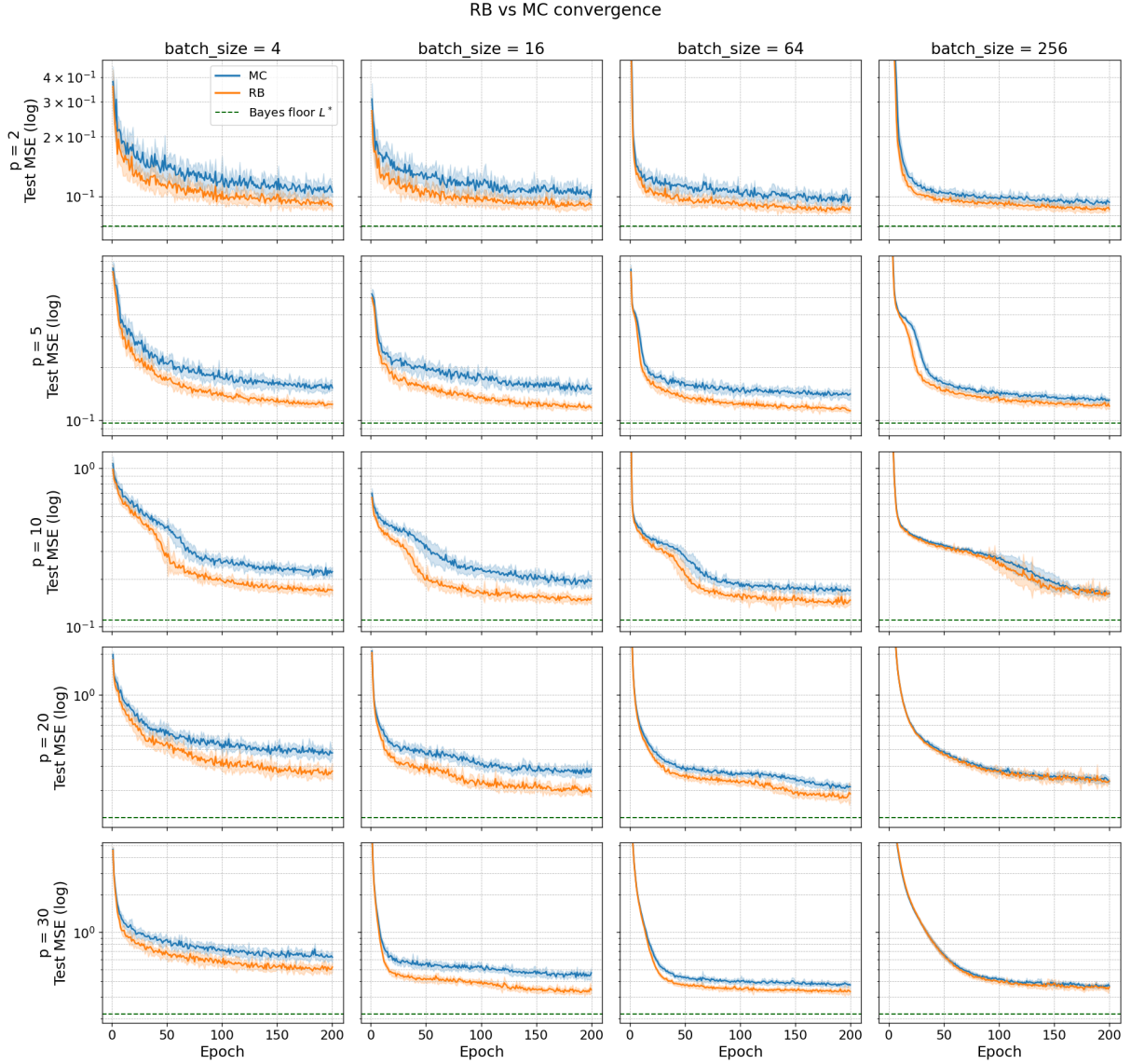


Figure 4.1: Training curves across parameter dimension $p \in \{2, 5, 10, 20, 30\}$ (rows) and batch size $\{4, 16, 64, 256\}$ (columns), at sample size $n = 50$ under the precision prior $\tau = \sigma^{-2} \sim \text{Uniform}(0.01, 1)$ (equivalently $\sigma \in (1, 10)$). Each panel shows the mean over seeds with a ± 1 standard-deviation band — Monte Carlo (MC) in blue, Rao-Blackwellised (RB) in orange. The dark green dashed line in each panel marks the Bayes-optimal floor $L^*(p)$ for that dimension. RB tracks at or below MC in every panel, at every batch size, and nearer the floor.

This pattern holds across the whole grid: the Rao-Blackwellised estimator attains a lower final loss than the Monte Carlo estimator in all twenty (p, batch) cells (Appendix B.2), and the advantage persists on a second, independent design matrix (Appendix B.5). This is consistent with the covariance calculation of Section 4.2.3. Increasing the parameter dimension adds coefficient directions to the sum (4.15); when these directions are weakly identified, they contribute additional posterior target variance to the Monte Carlo gradient. Increasing the batch size has the opposite effect on the observed gap: it does not change $\text{tr}(\Sigma_n)$ for an individual simulated problem, but it averages that target noise over more simulations, giving a $1/B$ scaling.

Approaching the Bayes floor. We can compare the neural estimators to the Bayes-optimal floor. At the baseline cell ($p = 20$, batch 16) the Monte Carlo estimator reaches a final loss of 0.276 and the Rao-Blackwellised estimator 0.198, against a floor of $L^* = 0.126$: the Rao-Blackwellised estimator closes about half (some 52%) of the gap between the Monte Carlo loss and the unreachable floor. The size of the gain varies across the grid with the excess-over-floor ratio $(\mathcal{L}_{\text{MC}} - L^*)/(\mathcal{L}_{\text{RB}} - L^*)$, which runs from near one at the largest batch, where both estimators have nearly exhausted the Monte Carlo noise, to above two at the smaller batches (Appendix B.2).

In some of the training curves, we observe both training curves coming asymptotically to different levels. Although, RB and MC have the same population asymptote, under finite-batch fixed-step optimisation they can have different effective asymptotes; RB’s is lower because it removes a positive semi-definite component of the gradient covariance. This phenomenon could be minimised by applying a *learning rate scheduler*, a standard neural network technique which dynamically adjusts the learning rate as training progresses.

Rao-Blackwellisation versus larger batches. Both Rao-Blackwellisation and a larger mini-batch reduce the noise in the training gradient (the former by removing the sampled-coefficient variance analytically, the latter by averaging it over more samples), so it is natural to ask whether Rao-Blackwellisation is simply a larger batch in disguise. A batch-size sweep at $p = 20$ over $\{4, 16, 64, 256\}$ shows that the two are related but not identical. The Rao-Blackwellised estimator at batch 4 matches the Monte Carlo estimator at batch 16: its analytic variance reduction is worth a fourfold increase in batch size. At every batch size its excess over the floor is the smaller of the two, and only at the largest batch, where both estimators have nearly exhausted the Monte Carlo noise, do they nearly coincide (Appendix B.3).

The cleanest way to see the gain is to plot the test error against the total number of simulated samples seen, rather than against epochs (Figure 4.2). Because each gradient step consumes a fresh batch of simulations, and simulation is the dominant cost of training, samples seen is the natural measure of computational effort. At every dimension and every batch size the Rao-Blackwellised estimator (solid) tracks below the Monte Carlo estimator (dashed): it reaches any given accuracy from fewer simulated samples. The advantage is a genuine saving in simulation, not an artefact of how the budget is split into steps.

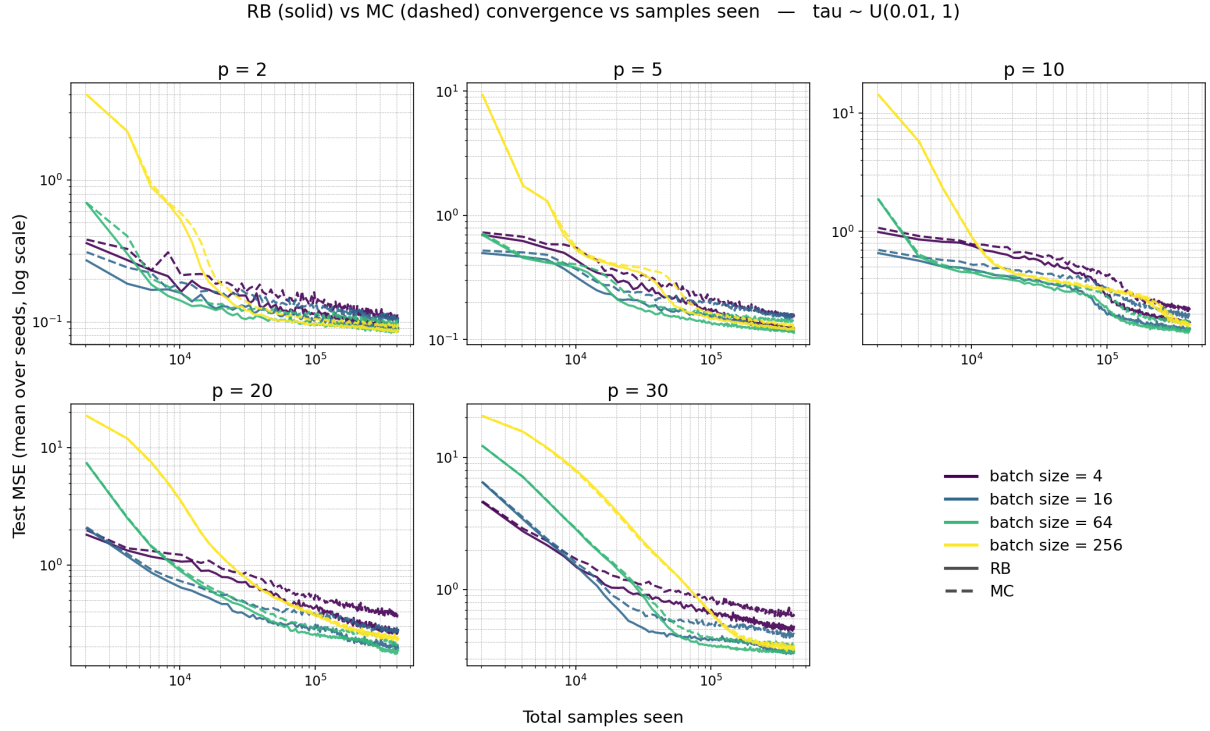


Figure 4.2: Test MSE against the total number of simulated samples seen (both axes logarithmic), for parameter dimension $p \in \{2, 5, 10, 20, 30\}$ and batch sizes $\{4, 16, 64, 256\}$ (colours), under $\tau \sim \text{Uniform}(0.01, 1)$. The Rao-Blackwellised estimator (solid) lies below the Monte Carlo estimator (dashed) at every batch size: it reaches a given accuracy from fewer simulated samples, a saving in the dominant cost of training.

4.3 Plug-in versus Rao-Blackwellised estimators

Our above analysis of the Rao-Blackwellised loss is built on the existence of an analytical conditional, in the case of linear regression this is $\beta_n(\sigma, \mathbf{y}) = \mathbb{E}[\beta | \mathbf{y}, \sigma]$. Since this conditional mean is available in closed form, it is natural to ask whether the network needs a β -head at all: one could train it to estimate only the noise scale $\hat{\sigma}_\gamma(\mathbf{y}) \approx \mathbb{E}[\sigma | \mathbf{y}]$ and recover the coefficients by substitution. This is the *plug-in* estimator,

$$\tilde{\beta}(\mathbf{y}) = \beta_n(\hat{\sigma}_\gamma(\mathbf{y}), \mathbf{y}),$$

the conditional mean evaluated at a point estimate of σ . While this estimator will also remove the noise introduced from sampling β , as we will now show, it is biased.

Let us first define the true posterior mean of the noise scale as $\bar{\sigma} = \mathbb{E}[\sigma | \mathbf{y}]$ and note the first two moments of the posterior of σ :

$$\mathbb{E}_{\sigma | \mathbf{y}}[\sigma - \bar{\sigma}] = 0, \quad \mathbb{E}_{\sigma | \mathbf{y}}[(\sigma - \bar{\sigma})^2] = \text{Var}(\sigma | \mathbf{y}).$$

Now let us fix $\mathbf{y}$ and expand $\beta_n(\sigma, \mathbf{y})$ to second order about $\bar{\sigma}$:

$$\beta_n(\sigma) = \beta_n(\bar{\sigma}) + \beta'_n(\bar{\sigma})(\sigma - \bar{\sigma}) + \frac{1}{2} \beta''_n(\bar{\sigma})(\sigma - \bar{\sigma})^2 + \dots \quad (4.17)$$

Using (4.3) and (4.17), the bias can be approximated as:

$$\mathbb{E}[\beta | \mathbf{y}] - \tilde{\beta}(\mathbf{y}) = \mathbb{E}_{\sigma | \mathbf{y}}[\beta_n(\sigma)] - \beta_n(\bar{\sigma}) \approx \frac{1}{2} \beta_n''(\bar{\sigma}, \mathbf{y}) \text{Var}(\sigma | \mathbf{y}), \quad (4.18)$$

dropping higher-order terms and treating $\hat{\sigma} = \bar{\sigma}$ to isolate the substitution error. The bias vanishes when β_n is affine in σ or the posterior of σ is degenerate, and otherwise scales with the curvature of β_n and the posterior variance of σ : it is the cost of conditioning on a point estimate of σ rather than integrating it out.

In the Bayesian linear regression model that curvature is genuinely present. The conditional mean

$$\beta_n(\sigma, \mathbf{y}) = (\sigma_0^{-2} I_p + \sigma^{-2} X^\top X)^{-1} \sigma^{-2} X^\top \mathbf{y}$$

is a nonlinear, saturating function of the noise scale — approaching the least-squares solution $(X^\top X)^{-1} X^\top \mathbf{y}$ as $\sigma \rightarrow 0$ and shrinking towards the prior mean 0 as σ grows — so $\beta_n'' \neq 0$ and (4.18) does not vanish.

The Rao-Blackwellised estimator does not pay this price, because its β -head performs the average in (4.5) for free. Its target is the posterior mean $\mathbb{E}[\beta | \mathbf{y}]$ — the same as the standard Monte Carlo head, with which it shares a minimiser by construction (Section 4.1.2) — and squared-error training realises that minimiser as the conditional mean of the target given $\mathbf{y}$. The marginalisation over σ that the plug-in skips is carried out by the regression itself: across training each $\mathbf{y}$ is paired with the spread of σ its posterior allows, and the fitted value is their average; a point estimate $\hat{\sigma}$ carries none of that spread. How much the plug-in loses therefore depends on how sharply the posterior of σ concentrates — small where σ is well identified, larger where it is not.

4.4 Discussion

The experiments support a strong conclusion, that Rao-Blackwellisation of the Bayes risk loss does indeed improve training efficiency. The Monte Carlo estimator carries the full gradient variance, including the large component contributed by the sampled coefficients β in the high-noise regime; the Rao-Blackwellised estimator removes that component analytically, substituting the conditional posterior mean, and settles markedly lower. At the baseline cell the Monte Carlo estimator’s final loss sits 119% above the floor and the Rao-Blackwellised estimator’s only 57% above it, closing about half of the Monte Carlo estimator’s excess.

This is the behaviour Section 4.1 predicts. Its gradient-variance result guarantees that Rao-Blackwellisation cannot raise the variance of the training gradient, for any model and any architecture, and a lower-variance gradient is a cleaner signal for the optimiser. The grid bears this out without exception: across every combination of seed, parameter dimension, and batch size, the Rao-Blackwellised estimator reached a lower final loss than the Monte Carlo estimator.

Section 4.1.5 shows that when the network is linear in its weights, Rao-Blackwellisation reduces the variance not only of the gradient but of the fitted weights themselves; for a nonlinear network

no such closed-form statement is available. The estimator trained here is nonlinear, a multilayer perceptron, so that stronger weight-level guarantee does not formally apply to it. That Rao-Blackwellisation nonetheless lowers its loss in every cell is empirical evidence that the linear-case mechanism carries over, consistent with reading the final layer of the network as a linear map of fixed features, the regime in which Section 4.1.5 does apply.

Chapter 5

Importance Sampling

Chapter 4 reduced the variance of the training gradient by integrating a conjugate block of the parameter vector out of the loss. That construction came with a guarantee. The Rao-Blackwell variance gap (4.9) is positive semi-definite, so the construction can never make the gradient noisier, for any model and any architecture; on Bayesian linear regression it gave faster convergence and a lower error in every cell of the grid. This chapter asks whether a second Monte Carlo variance-reduction technique, *importance sampling*, can help training in the same way.

The idea came from an experiment rather than from theory. While testing estimator architectures on a first-order autoregressive model, we compared the trained network against an exact-posterior reference, and its accuracy looked uneven across the parameter space. The network seemed to do worst on the near-non-stationary series at the edges of the autoregressive coefficient, and on the faint, low-amplitude series at small noise scale. These are the regions where the realisations are most distinctive and the process is hardest to simulate, so we suspected they were also the hardest to estimate. Under a uniform prior, though, the network sees them no more often than any other part of the parameter space. Importance sampling offered a way to change that: draw the training parameters from a proposal that oversamples the difficult regions, then reweight the loss so that it stays unbiased for the original prior. If the network’s error were concentrated in those regions, spending more of the simulation budget there should buy accuracy for the same number of training steps.

The result is negative. Across a family of proposals that oversample the edges of the parameter space ever more aggressively, the test error of the trained estimator does not fall; under the most aggressive proposal it rises, while the effective sample size collapses by more than an order of magnitude. The negative result is worth reporting because its cause is specific, and because it contrasts with the previous chapter. Section 5.2.5 traces the cause to two facts about the experiment: the trained estimator is already close to the exact posterior across the whole prior, so little reducible error remains for any proposal to recover, and the regions we oversampled are not the ones where its error is largest. Section 5.3 sets out the conditions under which importance sampling does help training, and why none of them held here. Rao-Blackwellisation

never makes training worse, for any model and architecture; importance sampling helps only when the proposal matches the structure of the problem, and when it does not, the best it can do is leave the error unchanged.

5.1 Importance sampling

Importance sampling is a Monte Carlo technique for estimating an expectation $\mathbb{E}_{\boldsymbol{\theta} \sim p}[f(\boldsymbol{\theta})]$ by drawing from a different distribution q and reweighting:

$$\mathbb{E}_{\boldsymbol{\theta} \sim p}[f(\boldsymbol{\theta})] = \int f(\boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta} = \int f(\boldsymbol{\theta}) \frac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})} q(\boldsymbol{\theta}) d\boldsymbol{\theta} = \mathbb{E}_{\boldsymbol{\theta} \sim q} \left[f(\boldsymbol{\theta}) \frac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})} \right].$$

The two Monte Carlo estimators,

$$\begin{aligned} \hat{L}_{\text{MC}} &= \frac{1}{N} \sum_{i=1}^N f(\boldsymbol{\theta}_i), & \boldsymbol{\theta}_i &\sim p(\boldsymbol{\theta}), \\ \hat{L}_{\text{IS}} &= \frac{1}{N} \sum_{i=1}^N w(\boldsymbol{\theta}_i) f(\boldsymbol{\theta}_i), & \boldsymbol{\theta}_i &\sim q(\boldsymbol{\theta}), \end{aligned}$$

where $w(\boldsymbol{\theta}) = p(\boldsymbol{\theta})/q(\boldsymbol{\theta})$ is the *importance weight*, are both unbiased for the same expectation. The reweighted estimator $\hat{L}_{\text{IS}}$ can have lower variance than the ordinary estimator $\hat{L}_{\text{MC}}$ when

$$\text{Var}_{\boldsymbol{\theta} \sim q}[w(\boldsymbol{\theta}) f(\boldsymbol{\theta})] < \text{Var}_{\boldsymbol{\theta} \sim p}[f(\boldsymbol{\theta})].$$

For a non-negative integrand the proposal that minimises the variance of $\hat{L}_{\text{IS}}$ is

$$q^*(\boldsymbol{\theta}) \propto f(\boldsymbol{\theta}) p(\boldsymbol{\theta}), \tag{5.1}$$

which follows from a Cauchy–Schwarz bound on the second moment of the reweighted integrand (Appendix A.2); under this proposal the weighted integrand $w(\boldsymbol{\theta})f(\boldsymbol{\theta})$ is constant and the variance is zero. Equation (5.1) says to put the proposal mass where the integrand is large, which is exactly the intuition behind oversampling the hard regions. Section 5.3 returns to this to ask which problems importance sampling can help; for now it serves as motivation.

In practice the normalising constants of p and q may be unknown, and the *self-normalised* estimator

$$\hat{r}_{\text{SNIS}}(\boldsymbol{\gamma}) = \frac{\sum_{i=1}^N w(\boldsymbol{\theta}_i) L(\boldsymbol{\theta}_i, \hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}_i))}{\sum_{i=1}^N w(\boldsymbol{\theta}_i)} \tag{5.2}$$

is used instead; it is biased at finite N but consistent. To monitor what a proposal costs we track its *effective sample size*

$$\text{ESS} = \frac{(\sum_i w(\boldsymbol{\theta}_i))^2}{\sum_i w(\boldsymbol{\theta}_i)^2} = \left(\sum_i \tilde{w}_i^2 \right)^{-1}, \tag{5.3}$$

where $\tilde{w}_i = w(\boldsymbol{\theta}_i)/\sum_j w(\boldsymbol{\theta}_j)$ are the normalised weights. It lies between 1 and N and counts, loosely, how many of the N reweighted draws carry real weight in the sum. Strictly, this is

the Kish effective sample size of the self-normalised mean: a convenient proxy for the noise a proposal injects rather than an exact count of the gradient’s effective draws, which Section 5.2.4 measures directly. Still, it captures the price of tilting the proposal away from the prior: when the effective sample size falls well below N , a few draws dominate the sum, a warning sign that the proposal has been pushed too far from the prior and may inject more gradient noise than its reweighting removes.

Training a neural Bayes estimator is itself a Monte Carlo integration: the population loss is the Bayes risk $r(\gamma) = \mathbb{E}_{\boldsymbol{\theta}, \mathbf{y}}[L(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}))]$, and each minibatch estimates its gradient by drawing parameters from the prior. Drawing $\boldsymbol{\theta}$ from a proposal q and reweighting gives the same risk,

$$r(\gamma) = \int p(\boldsymbol{\theta}) \mathbb{E}_{\mathbf{y}|\boldsymbol{\theta}}[L] d\boldsymbol{\theta} = \int \frac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})} q(\boldsymbol{\theta}) \mathbb{E}_{\mathbf{y}|\boldsymbol{\theta}}[L] d\boldsymbol{\theta} = \mathbb{E}_{\boldsymbol{\theta} \sim q}[w(\boldsymbol{\theta}) \mathbb{E}_{\mathbf{y}|\boldsymbol{\theta}}[L]]. \quad (5.4)$$

The reweighted objective on the right of (5.4) has the same expectation as the original for every proposal q , so the trained estimator still targets the posterior under the original prior p ; reweighting only concentrates the simulation budget where q places mass. The same identity holds for the gradient that stochastic gradient descent actually uses, so a proposal changes which parameter values the gradient is averaged over, not what it estimates. Section 5.3 returns to the question of when this helps.

5.2 Application to an AR(1) model

5.2.1 The AR(1) model and a recurrent estimator

The test model is the first-order autoregressive process of Example 3, equation (2.11): $x_t = \rho x_{t-1} + \varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, estimand $\boldsymbol{\theta} = (\rho, \sigma)$, priors $\rho \sim \text{Uniform}(-1, 1)$ and $\sigma \sim \text{Uniform}(0, A)$, and the process initialised from its stationary distribution $\mathcal{N}(0, \sigma^2/(1 - \rho^2))$. As $|\rho| \rightarrow 1$ the process approaches non-stationarity, and at $\rho = 1$ it is a random walk.

Figure 5.1 shows how the realisations change across the prior. Persistence grows with the autoregressive coefficient: at $\rho = 0$ the series is white noise, while near $|\rho| = 1$ it wanders far from the mean, and the noise scale σ sets the amplitude. The corners of the (ρ, σ) box thus produce qualitatively different, *harder-looking* series than the bulk of the prior. As the introduction noted, these corners look like where the estimator should struggle, which makes them the obvious place to spend extra training draws; Section 5.2.5 shows that for ρ the appearance is deceptive.

Because the data (2.11) are a time series rather than a set of exchangeable observations, the permutation-invariant DeepSets architecture of Section 3.3.1 is inappropriate. The estimator is instead a gated recurrent unit [7]: a recurrent encoder processes the series, its hidden states are mean pooled, and a small multilayer perceptron maps the pooled representation to $(\hat{\rho}, \hat{\sigma})$, with a tanh output on the ρ -head and a softplus output on the σ -head to enforce the parameter constraints. The estimator is trained under squared-error loss, so by Proposition 3.2 it targets the posterior mean. The series is fed to the encoder on its raw scale, without standardisation: the amplitude of an AR(1) realisation is set by σ , so rescaling each series to unit variance would erase the scale that identifies σ , whereas the raw series lets the network read σ off the amplitude

Example AR(1) realisations across the prior ($T = 100$, stationary start)

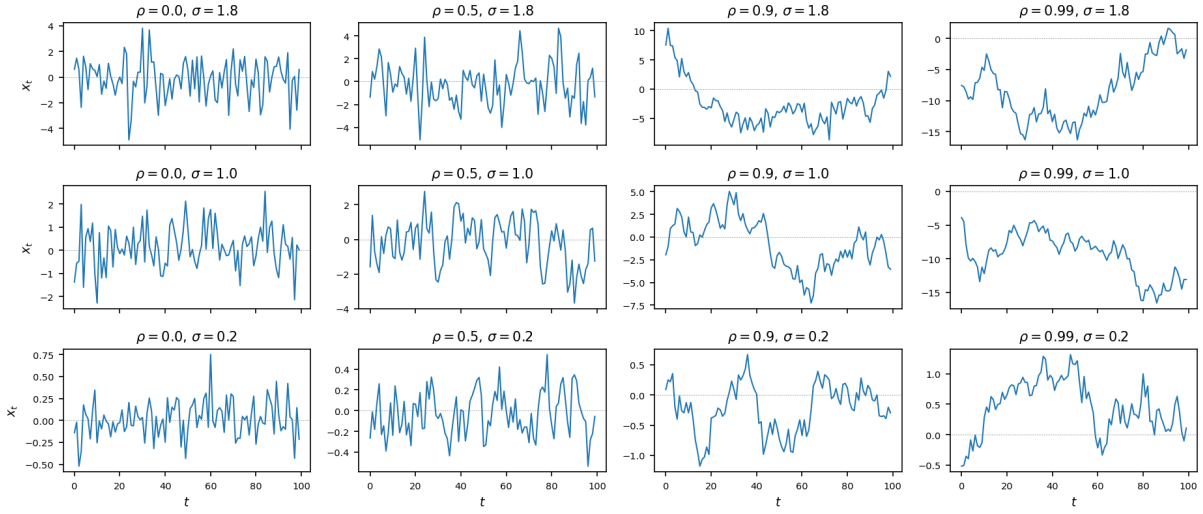


Figure 5.1: Example AR(1) realisations across the prior ($T = 100$, from the stationary distribution). Persistence grows with ρ (columns): white noise at $\rho = 0$, a near-random-walk at $\rho = 0.99$. The noise scale σ (rows) sets the amplitude.

and ρ off the autocorrelation. Both parameters are then identifiable from the input.

To judge the estimator we need a per-dataset gold standard, and the AR(1) model supplies one: it is conditionally conjugate in both parameters, so the Gibbs sampler of Section 2.3.3 draws from the exact posterior, and its post-burn-in average is the posterior-mean reference against which the trained estimator is judged. Comparing the two is what makes the loss interpretable: it separates the error the network could in principle train away from the error that any estimator on the same data must incur.

5.2.2 A family of proposals

With the goal of training our neural network to better fit the edges of the prior, we need a family of proposals that oversample those edges and a single knob that controls how aggressively. For a parameter with bounded support the Beta distribution provides exactly this. The $\text{Beta}(\alpha, \alpha)$ density on $[0, 1]$ is symmetric, and the single parameter α controls where it places mass: $\alpha = 1$ is uniform, $\alpha < 1$ is U-shaped and concentrates mass at the boundaries, and $\alpha > 1$ is bell-shaped and concentrates it in the centre. The proposals that target the difficult-looking boundary take $\alpha < 1$, with $\alpha = 1$ as the uniform baseline. We also include two proposals on the opposite tilt, at $\alpha = 1.5$ and $\alpha = 2$; what they do to each coordinate is set out below.

For the autoregressive coefficient, a draw $\tilde{\rho} \sim \text{Beta}(\alpha, \alpha)$ on $(0, 1)$ is mapped to $(-1, 1)$ by $\rho = 2\tilde{\rho} - 1$; by the change-of-variables formula the proposal density and importance weight are

$$q(\rho) = \frac{1}{2} \text{Beta}\left(\frac{\rho+1}{2}; \alpha, \alpha\right), \quad w(\rho) = \frac{p(\rho)}{q(\rho)} = \frac{1}{\text{Beta}\left(\frac{\rho+1}{2}; \alpha, \alpha\right)}. \quad (5.5)$$

At $\alpha = 1$ the Beta density is identically one and $w \equiv 1$, recovering uniform training. The noise

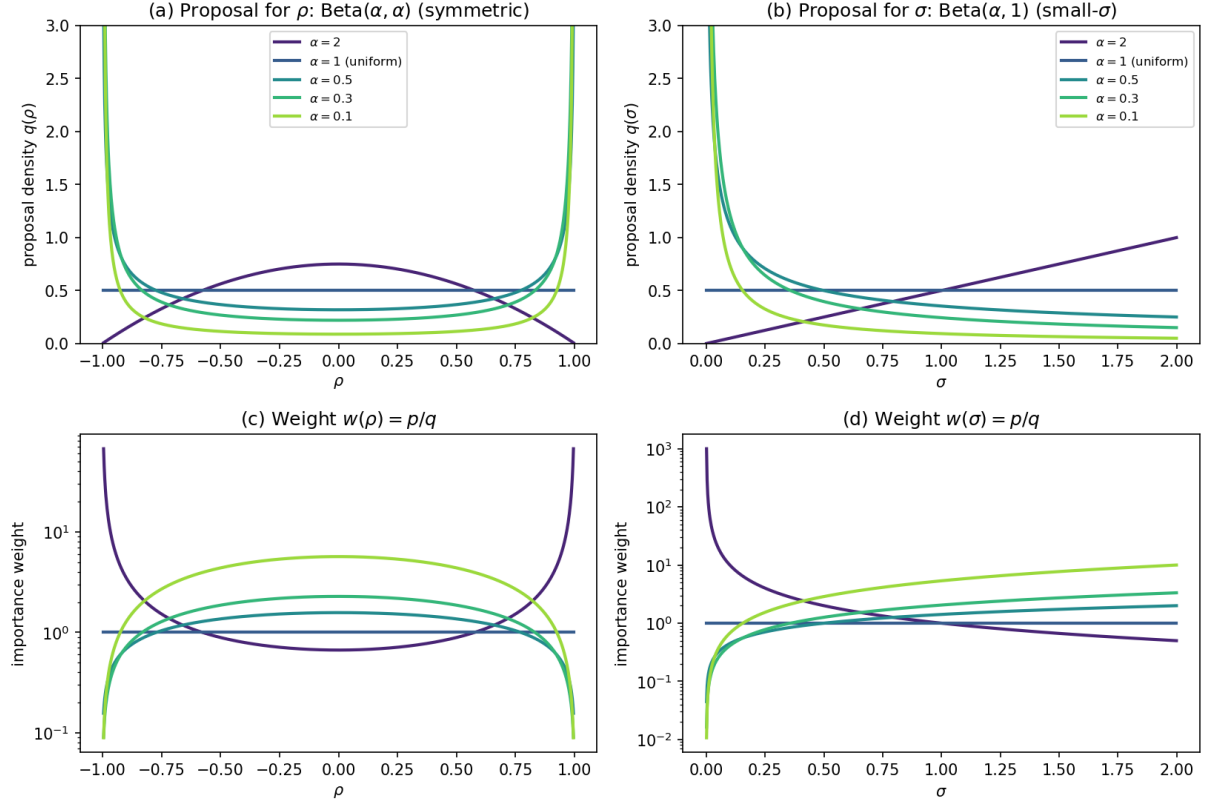


Figure 5.2: The Beta proposal family. **(a)** The ρ -proposal $\text{Beta}(\alpha, \alpha)$: $\alpha < 1$ oversamples the boundaries $|\rho| \rightarrow 1$, $\alpha > 1$ the interior, $\alpha = 1$ is uniform. **(b)** The σ -proposal $\text{Beta}(\alpha, 1)$: $\alpha < 1$ oversamples small σ , $\alpha > 1$ large σ . **(c, d)** The corresponding importance weights $w = p/q$.

scale is treated differently. Small σ gives a faint, near-white-noise series that looks hard to pin down, so we oversample only that low-noise tail, as a one-sided test of whether concentrating draws on a plausibly hard region buys accuracy there. An asymmetric proposal $\tilde{\sigma} \sim \text{Beta}(\alpha, 1)$, mapped by $\sigma = A\tilde{\sigma}$, does this:

$$q(\sigma) = \frac{1}{A} \text{Beta}\left(\frac{\sigma}{A}; \alpha, 1\right), \quad w(\sigma) = \frac{1}{\text{Beta}\left(\frac{\sigma}{A}; \alpha, 1\right)},$$

and the joint weight is the product of the two factors. Run at $\alpha > 1$ instead, the two proposals reverse: the ρ -proposal concentrates on the interior near $\rho = 0$ and the σ -proposal on large σ . We include two such cases, at $\alpha = 1.5$ and $\alpha = 2$, as the opposite tilt, to check whether aiming at those regions does any better than aiming at the boundary. Smaller α oversamples the targeted regions more aggressively but makes w more variable, which lowers the effective sample size through (5.3). That tension, between concentrating the draws and keeping enough of them informative, is what the sweep below probes. Figure 5.2 shows the mechanism: as α falls the ρ -proposal piles mass at both ends $|\rho| \rightarrow 1$ and the σ -proposal piles mass at small σ , while the compensating weights spread over more orders of magnitude. Figure 5.3 makes the same reshaping concrete in the (ρ, σ) plane.

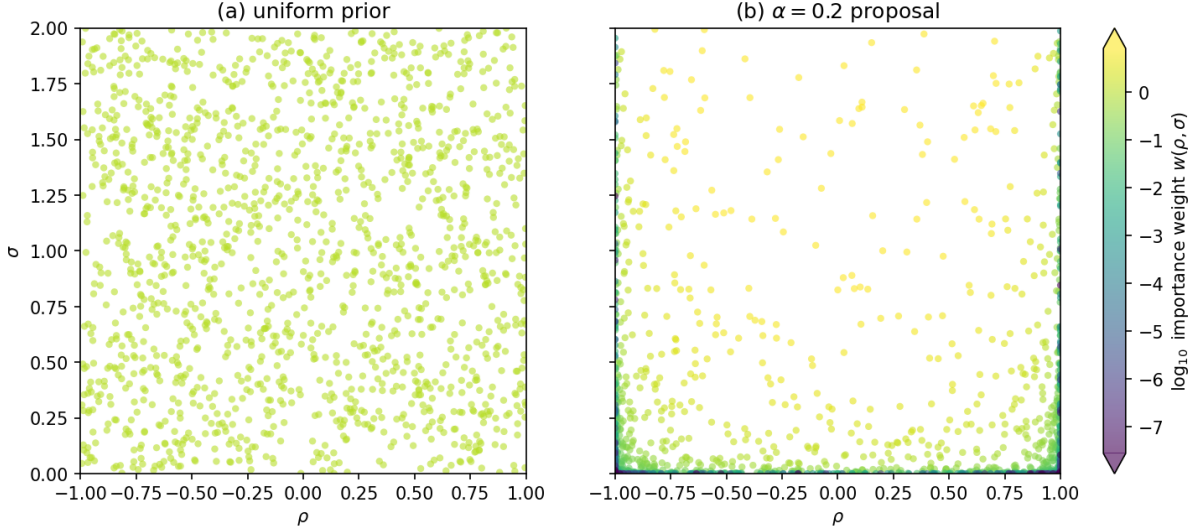


Figure 5.3: Training draws in the (ρ, σ) plane, coloured by importance weight w (log scale). **(a)** The uniform prior ($\alpha = 1$): even draws, unit weights. **(b)** An aggressive proposal ($\alpha = 0.2$): draws cluster on the $|\rho| \rightarrow 1$ edges and the small- σ floor (small weights, dark), leaving the interior and large σ sparse (large weights, bright).

Table 5.1: Test MSE by proposal strength α and region, with the mean effective sample size (batch size 256). Regions: the near-non-stationary boundary $|\rho| > 0.8$, low noise $\sigma < 0.25A$, and high noise $\sigma > 0.75A$.

α	all	$ \rho > 0.8$	low σ	high σ	ESS
2.0	0.0181	0.0196	0.0161	0.0245	93.1
1.5	0.0152	0.0123	0.0121	0.0198	172.7
1.0	0.0136	0.0100	0.0087	0.0187	256.0
0.5	0.0141	0.0102	0.0099	0.0188	155.9
0.3	0.0139	0.0100	0.0084	0.0181	78.8
0.1	0.0163	0.0128	0.0112	0.0217	13.4

5.2.3 Results

Estimators were trained at six proposal strengths, $\alpha \in \{2.0, 1.5, 1.0, 0.5, 0.3, 0.1\}$: the uniform baseline at $\alpha = 1$, three increasingly aggressive boundary proposals at $\alpha < 1$, and two proposals on the opposite tilt at $\alpha = 1.5$ and $\alpha = 2$. The test error was broken down over the whole prior and over three regions of interest: the near-non-stationary boundary $|\rho| > 0.8$, the low-noise region $\sigma < 0.25A$, and the high-noise region $\sigma > 0.75A$. Table 5.1 reports the mean squared error in each region together with the mean effective sample size at each α .

The result is negative. Across the whole prior the test error is essentially flat for the baseline and the two milder boundary proposals (0.0136, 0.0141, 0.0139) and rises at both extremes: climbing through $\alpha = 1.5$ (0.0152) to 0.0181 at $\alpha = 2$ on one side, and to 0.0163 at $\alpha = 0.1$ on the other. No proposal improves on the uniform baseline. The regional breakdown (Figure 5.4) shows why there is nothing to redistribute. The low- σ region the boundary proposals oversample is already the easiest at baseline (0.0087, the smallest of the four), and flooding it with draws leaves it there; the high- σ region they starve is the hardest (0.0187) and stays so; and the

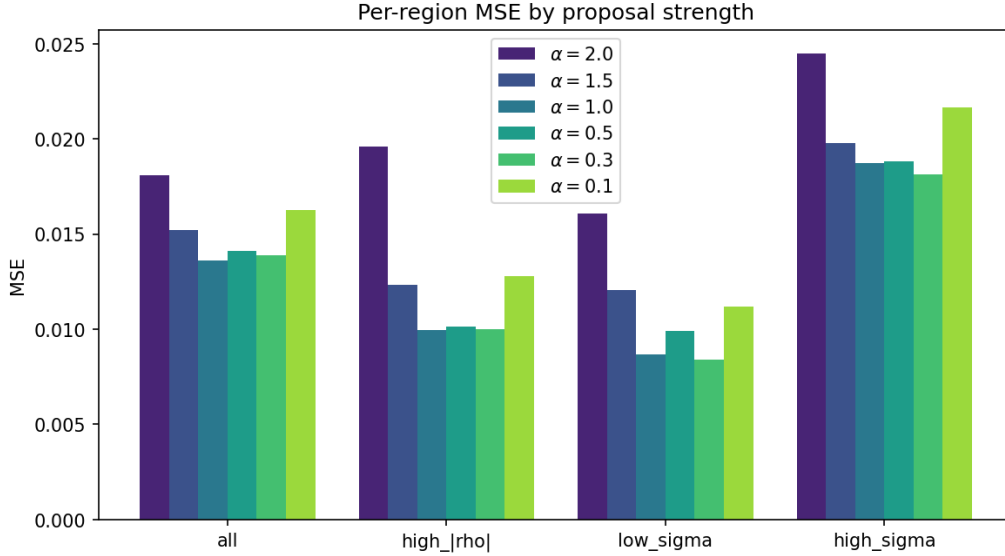


Figure 5.4: Test MSE by region for each proposal strength α . The baseline and the two milder boundary proposals ($\alpha = 1, 0.5, 0.3$) sit lowest and together; the interior tilt $\alpha = 1.5$ is intermediate, and the extremes $\alpha = 0.1$ and $\alpha = 2$ are worse in every region.

$|\rho| > 0.8$ boundary the ρ -proposal targets is among the regions where the ρ -error is smallest. The interior tilts at $\alpha = 1.5$ and $\alpha = 2$ aim the other way, towards the ρ -interior and high σ , and fare no better: their error is higher in every region, the high- σ region they now oversample included (0.0245 at $\alpha = 2$). The estimator already handles every region about as well as it can, so reweighting finds no slack to recover, whichever way it aims. At $\alpha = 0.1$ this is starkest: every region degrades together, the flooded low- σ tail included (it rises from 0.0087 to 0.0112), as the collapsed effective sample size leaves the gradient too noisy to train well.

What does fall is the effective sample size: it peaks at 256 for the uniform baseline and drops away as the proposal tilts in either direction, to 173 at $\alpha = 1.5$ and 93 at $\alpha = 2$ on one side and 13 at $\alpha = 0.1$ on the other, as the increasingly variable weights (5.5) let a handful of draws dominate the reweighted sum. This is the proposal's one reliable effect, and it is a cost, not a benefit. The training curves (Figure 5.5) show its consequence: the three milder proposals converge together near the floor, while $\alpha = 1.5$ settles a little above them and the two extremes, $\alpha = 0.1$ and $\alpha = 2$, sit higher still throughout, jitterier and settling at a higher loss rather than the same one.

5.2.4 The gradient variance, measured directly

The previous section read the negative result off two indirect quantities: the test error of the trained estimator, which no proposal lowered, and the effective sample size, which every proposal lowered. Neither is the quantity importance sampling is meant to reduce. What training actually consumes is the gradient: by (5.4) each minibatch estimates $\nabla_{\gamma} r$ by reweighting draws from the proposal, and the speed of stochastic gradient descent is governed by the variance of that estimate, not by the variance of the loss nor by the effective sample size on its own. This section measures the gradient variance directly, and so asks of each proposal the question the chapter

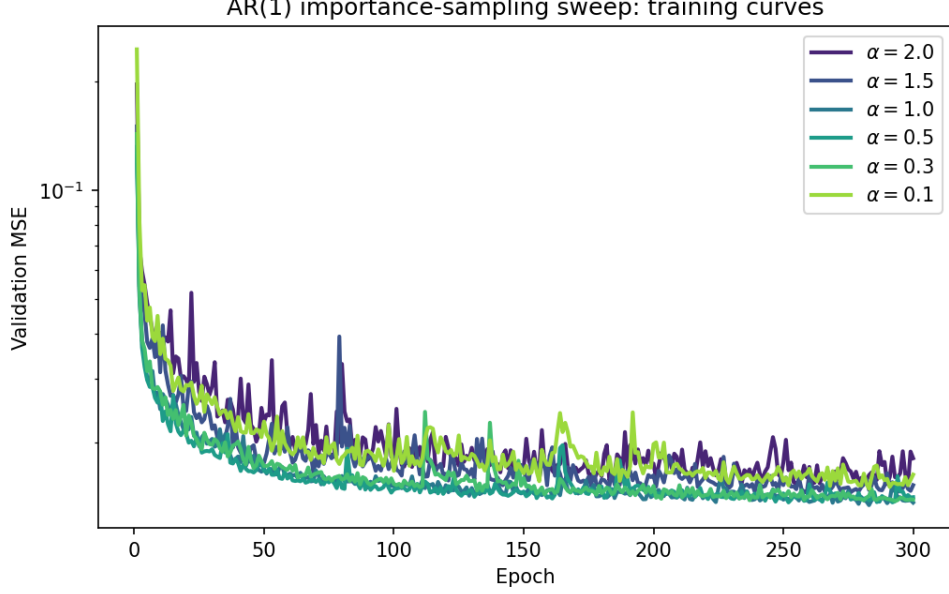


Figure 5.5: Seed-averaged validation MSE over training for the six proposals. The three milder settings ($\alpha = 1, 0.5, 0.3$) converge together near the floor; the interior tilt $\alpha = 1.5$ sits a little above them, and the extremes $\alpha = 0.1$ and $\alpha = 2$ stay higher still, jitterier and settling higher.

has thus far answered only by proxy: did reweighting make the gradient less noisy, or more?

The measurement holds the estimator fixed. At a frozen set of weights γ and a proposal q_α , a single minibatch of the common size $B = 256$ produces the self-normalised gradient

$$\mathbf{g} = \nabla_\gamma \sum_{i=1}^B \tilde{w}_i L(\theta_i, \hat{\theta}_\gamma(\mathbf{y}_i)), \quad \theta_i \sim q_\alpha,$$

exactly the estimator the training loop applies in (5.2). Drawing M independent minibatches, each of the same size B so that every α spends the identical simulation budget per step, the total gradient variance is estimated by

$$V(\alpha) = \text{tr}(\text{Cov}(\mathbf{g})) = \frac{1}{M} \sum_{m=1}^M \|\mathbf{g}^{(m)} - \bar{\mathbf{g}}\|_2^2, \quad \bar{\mathbf{g}} = \frac{1}{M} \sum_{m=1}^M \mathbf{g}^{(m)},$$

the spread of the minibatch gradient about its own mean, summed over every weight of the network. The quantity reported is the ratio $V(\alpha)/V(1)$ to the uniform baseline: a value below one is a genuine variance reduction, a value above one is added noise, and the baseline sits at one by construction. Because the variance depends on the weights at which it is taken, the comparison is made at the same network for every proposal: a fresh random initialisation, fixed and identical across the sweep. Each ratio is averaged over three measurement seeds.

The result is unambiguous (Figure 5.6). The gradient variance is smallest at the uniform baseline and rises under every proposal; no setting of α reduces it, and the inflation grows monotonically as the proposal tilts further from uniform in either direction. The milder proposals raise it only gently — by a factor of about 1.3 at $\alpha = 0.5$, 2.2 at $\alpha = 0.3$, and 2.3 at the opposite tilt $\alpha = 1.5$

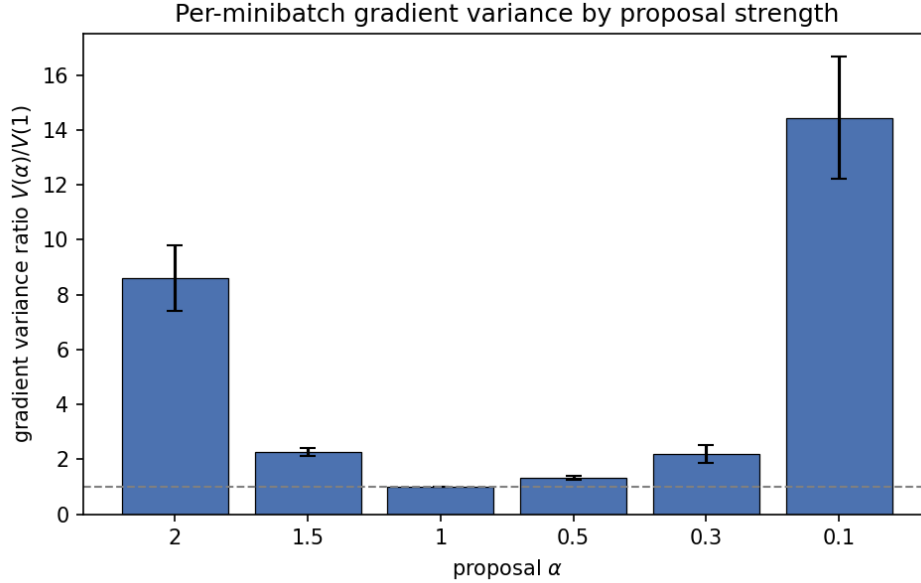


Figure 5.6: Per-minibatch gradient variance by proposal strength, as the ratio $V(\alpha)/V(1)$ to the uniform baseline, measured at a fresh random initialisation and averaged over three seeds (error bars show the seed spread). The dashed line marks the uniform baseline at one; every proposal lies above it, so none reduces the variance, and the two extremes ($\alpha = 0.1$ and $\alpha = 2$) inflate it by roughly an order of magnitude.

Table 5.2: Test MSE of the GRU estimator and the Gibbs posterior-mean (Bayes) reference, by parameter, on a held-out test set.

	MSE in ρ	MSE in σ
Gibbs posterior mean (Bayes)	0.0073	0.0056
GRU estimator	0.0089	0.0063

— while the extremes raise it sharply, by roughly $14\times$ at $\alpha = 0.1$ and $8.6\times$ at $\alpha = 2$. This is the direct counterpart of the test-error and effective-sample-size findings of Section 5.2.3: the one reliable effect of reweighting here is to make the gradient noisier. No proposal in the Beta family reduced the variance that training consumes; Section 5.2.5 explains why none could.

5.2.5 Why reweighting cannot help here

Reweighting does not change what the training objective is, only how noisily each minibatch estimates it (5.4), so it can speed training only by making the gradient less noisy. For that to buy anything, two things must hold: the estimator must have error left to remove, and the proposal must put its draws where that error is. On the AR(1) problem neither holds, and both facts come from comparing the trained network with the exact posterior.

The estimator is already about as accurate as any estimator could be. Table 5.2 compares the GRU with the Gibbs posterior mean, the Bayes-optimal estimator for this model, on the same test sets. The GRU’s error exceeds the Gibbs error by 22% on the autoregressive coefficient and 13% on the noise scale.

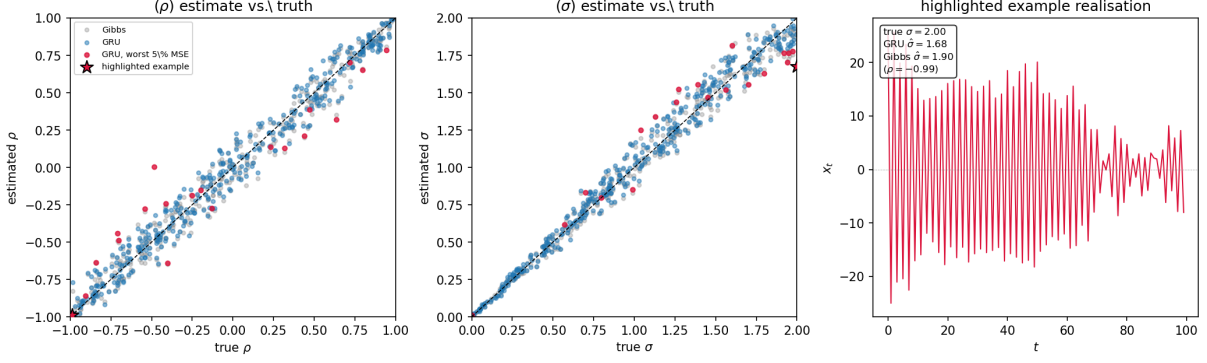


Figure 5.7: Estimates from the baseline GRU (blue) and the Gibbs posterior mean (grey) against the truth, for ρ (left) and σ (centre); the worst-MSE cases are red. Right: one of them as a realisation, an extreme series with $\rho \approx -0.99$ and σ at the top of its range. The GRU recovers ρ exactly, though the series looks the hardest of all; both it and Gibbs underestimate the extreme σ , so that residual is shared with the Bayes-optimal sampler, not something the network could train away.

The error on any one dataset has two parts. One is the posterior uncertainty that no estimator can remove: even the Bayes-optimal posterior mean misses, because the data do not pin the parameters down exactly. The other is the excess the network carries on top, the gap between it and that optimum. Reweighting cannot touch the first part, and it recovers the second only where training is held back by gradient noise, the one thing a cleaner gradient improves. That excess is small in absolute terms (Table 5.2), and Figure 5.7 shows what it is: on the hardest series the GRU and the Gibbs sampler miss together, so the error there is posterior uncertainty, not a deficit a better-aimed gradient could close. Little of the error anywhere is the kind a proposal could recover.

The little error that remains sits in the opposite place from where the proposals aim. Figure 5.8 bins the test error by each parameter. The error in ρ is largest in the interior, near $\rho = 0$, and smallest at the boundaries $|\rho| \rightarrow 1$, the very edge the ρ -proposal floods; it is also raised where σ is small. The error in σ runs the other way, growing with σ , so the high-noise end carries most of it while the low- σ tail the σ -proposal floods is the cleanest part.

There is a clean reason the boundary is the easy end for ρ . Estimating ρ is a regression of each x_t on its lag x_{t-1} , so the posterior variance of the slope is the noise variance over the spread of the predictor, $\text{Var}(\rho \mid \sigma^2, \mathbf{x}) = \sigma^2/S$ with $S = \sum_t x_{t-1}^2$ (Section 2.3.3). The series starts from its stationary distribution, of variance $\sigma^2/(1 - \rho^2)$, so $S \approx T\sigma^2/(1 - \rho^2)$ and the posterior variance is $\approx (1 - \rho^2)/T$. As $|\rho| \rightarrow 1$ the series makes larger, more persistent excursions, and this variance vanishes: the data pin ρ down most tightly exactly where the series looks most dramatic. The motivating intuition mistook one kind of difficulty for another: a near-non-stationary series has large swings that carry the most information about ρ , so it is the *easiest* to estimate ρ from. The hard-to-simulate end and the hard-to-estimate end are opposite.

The $\alpha = 2$ proposal puts its draws in the ρ -interior and at high σ , where the error genuinely is largest, and it is no better than the rest (Table 5.1), because the error there is the irreducible kind the Gibbs sampler shares. So no setting of the Beta family helps: where the error is large

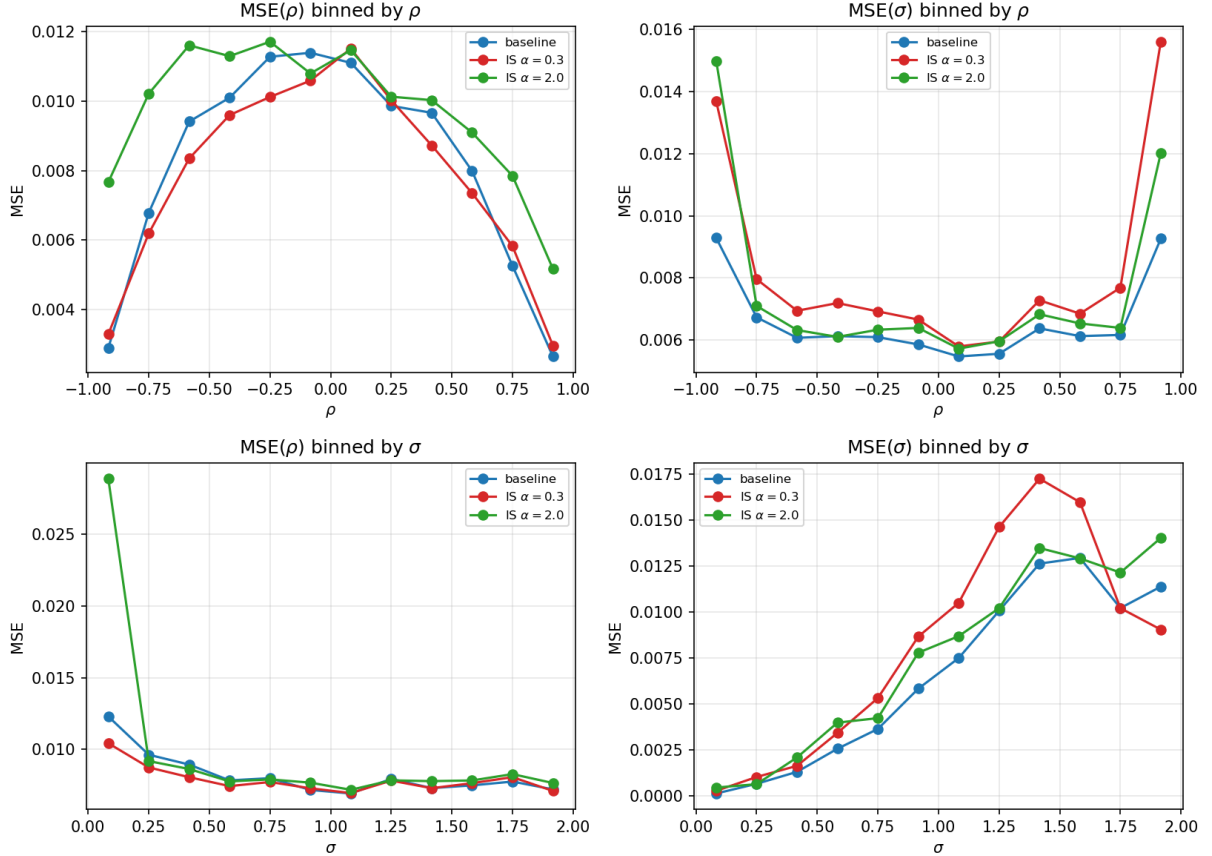


Figure 5.8: Test error binned by ρ (top row) and by σ (bottom row), for the baseline (blue), the boundary proposal $\alpha = 0.3$ (red), and the opposite tilt $\alpha = 2$ (green). The error lives in the ρ -interior and at large σ , not at the boundary or low- σ tail the proposals target; neither overlay improves on the baseline.

it cannot be removed, and where it can be removed it is already small.

Against these absent gains stands a cost that is always paid. The more a proposal concentrates its draws, the more variable its weights (5.5), the lower the effective sample size (5.3), and the noisier the gradient left by the self-normalised objective (5.2). With no gain to offset it, this cost is the whole effect: at $\alpha = 0.1$ it shows up as the across-the-board degradation of Section 5.2.3. Reweighting here can only add noise, and the harder it is pushed, the more it adds.

5.3 When importance sampling can help

The negative result is therefore a verdict on a mismatch, not on importance sampling in general. Three conditions separate the problems in which importance sampling can help from those in which it cannot.

The first is that there must be reducible error left to recover. Section 5.2.5 showed that little such error remains in the AR(1) experiment: the trained GRU is already close to the Gibbs posterior-mean reference. The remaining test error is therefore dominated by posterior uncertainty rather than by a failure of optimisation. Reweighting the training distribution cannot remove this

irreducible part of the Bayes risk.

The second condition is that the proposal must target where the useful gradient signal is concentrated, not merely where the raw loss is large. For squared-error loss,

$$L = \|\hat{\boldsymbol{\theta}}_{\gamma}(\mathbf{y}) - \boldsymbol{\theta}\|_2^2, \quad \nabla_{\gamma} L = 2 \left(\nabla_{\gamma} \hat{\boldsymbol{\theta}}_{\gamma}(\mathbf{y}) \right)^{\top} \left(\hat{\boldsymbol{\theta}}_{\gamma}(\mathbf{y}) - \boldsymbol{\theta} \right).$$

Thus, ignoring variation in the estimator sensitivity ($\|\nabla_{\gamma} \hat{\boldsymbol{\theta}}_{\gamma}(\mathbf{y})\|$), the gradient magnitude scales like $\sqrt{L}$, not like L itself. A proposal designed to reduce gradient variance should therefore be guided by the size of the gradient contribution, roughly $q \propto p\sqrt{L}$, rather than by the loss alone. More importantly, the relevant loss is the reducible excess over the Bayes estimator. Large loss caused by posterior uncertainty produces noisy residuals, but not a systematic gradient direction that the network can train away.

The third condition is that the variance affected by the proposal must be the dominant source of training noise. Importance sampling changes how often different parameter values $\boldsymbol{\theta}$ are drawn, but it does not remove the randomness from simulating a fresh dataset $(\mathbf{y} \mid \boldsymbol{\theta})$. If that conditional simulation noise dominates the minibatch gradient, then even a well-aimed proposal can only produce a limited gain.

The proposals used here were fixed in advance by the single parameter α , which cannot know where the recoverable gradient signal lies; on this problem it aimed badly. One direction for future work is to adapt the proposal during training, estimating where the gradient is largest as training proceeds and reshaping $q \propto p\sqrt{L}$ towards it, so that the simulation budget follows the signal rather than a fixed guess. Whether such a scheme can realise even the bounded gains these conditions allow, without its own overhead and instability, is an open question. A second is to test the method on a problem built to meet the conditions rather than to fail them, where the loss is sharply uneven, the estimator’s sensitivity varies in the same direction, and the residual error is genuinely reducible. That would be the fair positive counterpart to the AR(1) case study, and the place to further investigate whether importance sampling can meaningfully help neural Bayes training.

5.4 Discussion

The experiment returns a negative result, and the negative result is the point. On the AR(1) model, importance sampling with Beta proposals leaves the trained estimator’s test error unchanged for the milder proposals and raises it at both extremes, all while collapsing the effective sample size by more than an order of magnitude. Section 5.2.5 explains why: the estimator is already close to the Bayes reference everywhere, so little reducible error remains for any proposal to recover, and the regions the boundary proposals target are the ones the estimator already handles best. The motivating intuition confused the conspicuous regions, the ones that are dramatic to look at and awkward to simulate, with the difficult ones; for the autoregressive coefficient the two are opposite.

Section 5.3 turns that failure into a specification: importance sampling helps training only when there is reducible error to recover, when the proposal is aimed where the gradient signal actually concentrates, and when the simulation noise is not what limits training. A scale-estimation problem can be built to meet these conditions, where the gain is real if modest; the AR(1) study met none of them where its proposals aimed.

This is the genuine contrast with Chapter 4. The Rao-Blackwell variance gap (4.9) is positive semi-definite for every model and every architecture: the technique cannot make training worse, and on Bayesian linear regression it made training reliably and substantially better. Importance sampling carries no such guarantee. It helps only when the proposal is well matched and gradient noise is the binding constraint; when the difficulty lies elsewhere, as here, the best it can do is leave the error untouched, and pushed harder it makes the error worse, while its cost in effective sample size is paid either way. Both are instances of Monte Carlo variance reduction applied to estimator training, but they differ in reliability: one unconditional, the other contingent on a match between the proposal and the structure of the problem. That difference is the main lesson of this chapter.

Chapter 6

Discussion and Conclusion

This thesis began from a single observation: training a neural Bayes estimator involves Monte Carlo integration of the Bayes risk, and the gradient that drives training is therefore a Monte Carlo estimator with a variance. That variance is a property of how the loss is sampled, not of the inference problem, and it can be reduced.

The primary reduction attempted here is the Rao-Blackwellised loss. When a block of the parameter vector has a tractable conditional posterior, that block is integrated out of the loss analytically rather than sampled. For squared-error loss this amounts to substituting the conditional posterior mean for the sampled target, with the residual conditional variance entering only as a constant that does not affect the gradient. The resulting training gradient is the Rao-Blackwellisation of the standard one, and the law of total variance gives a positive semi-definite gradient-variance gap: the technique can never increase the variance of the training gradient. For a linear network the guarantee strengthens to the fitted weights themselves.

The empirical study on Bayesian linear regression bore this out. The Rao-Blackwellised estimator attained a lower loss than the standard estimator in every cell of the experimental grid, closed about half the gap between the standard estimator and the Bayes-optimal floor at the baseline, and matched the standard estimator’s accuracy at a quarter of the batch size.

Chapter 5 placed Rao-Blackwellisation alongside importance sampling, a second variance-reduction technique acting on the training of the estimator. The comparison is the clearest summary of what makes Rao-Blackwellisation attractive. Importance sampling can in principle sharpen estimation by oversampling the regions that contribute most to the loss, but only when the bottleneck on training is gradient noise and the proposal is matched to it. On the autoregressive model neither condition held: the trained estimator already sat close to the Bayes-optimal floor across the prior, and the proposal oversampled the very regions it handled best, so reweighting found no error worth recovering and, pushed hard, only added gradient noise. Rao-Blackwellisation carries an unconditional guarantee, the variance gap being positive semi-definite for every model and architecture, at the cost of requiring a tractable conditional posterior. Where that conditional structure is present, as it is in any conditionally conjugate hierarchical model, Rao-Blackwellisation is the safer and stronger of the two.

One distinction underlies this comparison and is worth stating directly. Reducing the variance of the loss — or, more precisely, of the gradient that drives training — is not really the primary goal; the goal is reducing the variance of the estimate that training produces, the fitted weights $\hat{\gamma}$ and the estimator they define. A cleaner gradient is valuable only insofar as it yields a less variable estimator. At this level the two techniques separate most sharply. For a linear network, Section 4.1.5 showed that Rao-Blackwellisation reduces not only the variance of the gradient but the variance of the fitted weights themselves, with the last-layer argument suggesting the same holds approximately for nonlinear networks. Importance sampling admits no such statement: it bears, and only contingently, on the variance with which a minibatch estimates the gradient, and nothing in it carries through to the variance of the estimator that training returns. That a guarantee can be carried to the fitted weights for one technique and not the other is the deepest form of the contrast between them.

Several limitations bound the results. The Rao-Blackwellised loss requires a tractable conditional posterior for the integrated block; where no parameter block is conditionally conjugate the construction does not apply, and only the sampled block benefits when one block is. The experimental evidence is confined to two models (linear regression with a multilayer perceptron, and an autoregressive process with a recurrent network) and does not establish how the method behaves for richer simulators or larger architectures. The estimators studied are point estimators and provide no quantification of posterior uncertainty.

6.1 Future work

Several directions follow naturally. The linear-network result of Section 4.1.5 suggests, but does not prove, a corresponding statement for fully nonlinear networks; making the last-layer argument rigorous, or bounding its error, would close that gap. The squared-error construction of Section 4.1.3 could be extended to other losses (the quantile loss of Section 3.1.1, for instance) to Rao-Blackwellise estimators of posterior quantiles rather than means. Where a parameter block is only approximately conjugate, a partial or approximate Rao-Blackwellisation may still reduce variance, and quantifying the resulting bias–variance trade-off is an open question. The variance-reduction toolbox of Monte Carlo contains further techniques not explored here (control variates, antithetic sampling, and stratified proposals among them), and a systematic comparison of these against Rao-Blackwellisation for estimator training, including how they might be combined, is a natural direction. A related direction is the reparameterisation of the estimator’s outputs to decouple weakly identified parameters, an alternative to variance reduction for the boundary difficulties of Chapter 5. Most importantly, the method should be tested on the larger, less tractable simulators that are the natural setting for simulation-based inference.

6.2 Conclusion

Neural Bayes estimators amortise Bayesian inference by replacing per-dataset sampling with a single trained mapping. Because their training is itself Monte Carlo integration, the variance-reduction toolbox of Monte Carlo applies to the training loop. This thesis has shown that

Rao-Blackwellisation, in particular, is a cheap and reliable lever: where a parameter block is conditionally conjugate, integrating it out of the loss provably reduces the variance of the training gradient, and in the regimes studied it drives the trained estimator substantially closer to the Bayes-optimal floor. Conditional conjugacy, long exploited to make Gibbs sampling possible, turns out to be equally useful for training the estimators that aim to replace it.

Appendix A

Derivations

A.1 Detailed balance for Metropolis–Hastings

This appendix verifies the claim of Section 2.2.1 that the Metropolis–Hastings chain satisfies detailed balance with respect to the posterior $\pi(\boldsymbol{\theta}) = p(\boldsymbol{\theta} | \mathbf{y})$. The transition kernel has an accepted-move component and a rejected-move component,

$$T(\boldsymbol{\theta}' | \boldsymbol{\theta}) = q(\boldsymbol{\theta}' | \boldsymbol{\theta}) \alpha(\boldsymbol{\theta}, \boldsymbol{\theta}') + \delta(\boldsymbol{\theta}' - \boldsymbol{\theta}) r(\boldsymbol{\theta}),$$

where δ is the Dirac delta and $r(\boldsymbol{\theta}) = \int [1 - \alpha(\boldsymbol{\theta}, \boldsymbol{\theta}'')] q(\boldsymbol{\theta}'' | \boldsymbol{\theta}) d\boldsymbol{\theta}''$ is the total rejection probability. For $\boldsymbol{\theta}' = \boldsymbol{\theta}$ detailed balance holds trivially, so take $\boldsymbol{\theta}' \neq \boldsymbol{\theta}$, where the delta term vanishes. Substituting the acceptance probability (2.9),

$$\begin{aligned} T(\boldsymbol{\theta}' | \boldsymbol{\theta}) \pi(\boldsymbol{\theta}) &= q(\boldsymbol{\theta}' | \boldsymbol{\theta}) \pi(\boldsymbol{\theta}) \min\left(1, \frac{\pi(\boldsymbol{\theta}') q(\boldsymbol{\theta} | \boldsymbol{\theta}')}{\pi(\boldsymbol{\theta}) q(\boldsymbol{\theta}' | \boldsymbol{\theta})}\right) \\ &= \min\left(q(\boldsymbol{\theta}' | \boldsymbol{\theta}) \pi(\boldsymbol{\theta}), q(\boldsymbol{\theta} | \boldsymbol{\theta}') \pi(\boldsymbol{\theta}')\right) \\ &= q(\boldsymbol{\theta} | \boldsymbol{\theta}') \pi(\boldsymbol{\theta}') \min\left(\frac{\pi(\boldsymbol{\theta}) q(\boldsymbol{\theta}' | \boldsymbol{\theta})}{\pi(\boldsymbol{\theta}') q(\boldsymbol{\theta} | \boldsymbol{\theta}')}, 1\right) \\ &= T(\boldsymbol{\theta} | \boldsymbol{\theta}') \pi(\boldsymbol{\theta}'), \end{aligned}$$

where the second equality moves the positive factor $q(\boldsymbol{\theta}' | \boldsymbol{\theta}) \pi(\boldsymbol{\theta})$ inside the minimum and the third moves the positive factor $q(\boldsymbol{\theta} | \boldsymbol{\theta}') \pi(\boldsymbol{\theta}')$ back out. The second line is symmetric under exchange of $\boldsymbol{\theta}$ and $\boldsymbol{\theta}'$, which is what the third line exploits. Detailed balance holds, so π is the stationary distribution of the chain.

A.2 The optimal importance sampling proposal

This appendix proves the claim of Section 5.1 that, among all proposals q , the variance of the importance sampling estimator $\hat{L}_{\text{IS}} = \frac{1}{N} \sum_i w(\boldsymbol{\theta}_i) f(\boldsymbol{\theta}_i)$, with weight $w(\boldsymbol{\theta}) = p(\boldsymbol{\theta})/q(\boldsymbol{\theta})$, is minimised by

$$q^*(\boldsymbol{\theta}) \propto |f(\boldsymbol{\theta})| p(\boldsymbol{\theta}).$$

Proof. Since $\hat{L}_{\text{IS}}$ is unbiased for every q , minimising its variance is equivalent to minimising the second moment of a single term,

$$\mathbb{E}_q \left[\left(\frac{f(\boldsymbol{\theta})p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})} \right)^2 \right] = \int \frac{f(\boldsymbol{\theta})^2 p(\boldsymbol{\theta})^2}{q(\boldsymbol{\theta})} \, \mathrm{d}\boldsymbol{\theta}.$$

By the Cauchy–Schwarz inequality, for any density q ,

$$\int \frac{f^2 p^2}{q} \, \mathrm{d}\boldsymbol{\theta} = \left(\int \frac{f^2 p^2}{q} \, \mathrm{d}\boldsymbol{\theta} \right) \left(\int q \, \mathrm{d}\boldsymbol{\theta} \right) \geq \left(\int |f| p \, \mathrm{d}\boldsymbol{\theta} \right)^2,$$

with equality if and only if $f^2 p^2 / q \propto q$, that is,

$$q^*(\boldsymbol{\theta}) \propto |f(\boldsymbol{\theta})| p(\boldsymbol{\theta}).$$

The lower bound $\left(\int |f| p \, \mathrm{d}\boldsymbol{\theta} \right)^2$ is independent of q , so q^* attains it and is therefore optimal. When $f \geq 0$ the resulting weighted integrand $w(\boldsymbol{\theta})f(\boldsymbol{\theta})$ is constant, and the variance is zero.

Appendix B

Experimental details

This appendix records the configuration and per-cell results of the Bayesian linear regression experiment of Chapter 4.

B.1 Configuration and protocol

The model is the hierarchical linear regression of equation (2.4), with prior scale $\sigma_0 = 5$ and noise-precision prior $\tau \sim \text{Uniform}(0.01, 1)$, placing the noise standard deviation in $\sigma \in (1, 10)$. The estimator is a multilayer perceptron of depth 3 and hidden width 128 with ReLU activations and a softplus output on the σ -head, trained with Adam at learning rate 10^{-3} .

The grid sweeps parameter dimension $p \in \{2, 5, 10, 20, 30\}$ and minibatch size $B \in \{4, 16, 64, 256\}$ at sample size $n = 50$, with a single design matrix fixed per dimension (a second, independent realisation is examined in Appendix B.5). Each cell is trained for 200 epochs over ten seeds, the Monte Carlo and Rao-Blackwellised estimators sharing their initialisation and their simulated batches. Each Bayes-optimal floor is computed by the Gibbs sampler of Section 4.2.4 on a held-out test set of 1024 problems; its variance-based and squared-error estimators agree at every dimension (Table B.3).

B.2 Main grid results

Table B.1 reports, for every (p, B) cell, the final test loss of each estimator (the mean of the last ten epochs, averaged over ten seeds), the Bayes floor L^* , the raw ratio $\mathcal{L}_{\text{MC}}/\mathcal{L}_{\text{RB}}$, and the excess-over-floor ratio $(\mathcal{L}_{\text{MC}} - L^*)/(\mathcal{L}_{\text{RB}} - L^*)$. The Rao-Blackwellised estimator attains the lower loss in all twenty cells.

B.3 Batch-size sweep

Table B.2 reports the $p = 20$ cell at batch sizes $B \in \{4, 16, 64, 256\}$ (ten seeds each, 200 epochs). The loss is the mean over the last ten epochs; the excess is that loss minus the floor $L^* = 0.126$.

Table B.1: Main grid: final test loss of the Monte Carlo (MC) and Rao-Blackwellised (RB) estimators, the Bayes floor L^* , and the raw and excess-over-floor ratios, for each (p, B) cell at $n = 50$.

p	B	$\mathcal{L}_{\text{MC}}$	$\mathcal{L}_{\text{RB}}$	L^*	ratio	excess ratio
2	4	0.1080	0.0921	0.0708	1.172	1.74
2	16	0.1033	0.0906	0.0708	1.140	1.64
2	64	0.0979	0.0868	0.0708	1.129	1.70
2	256	0.0932	0.0864	0.0708	1.079	1.43
5	4	0.1549	0.1234	0.0964	1.255	2.16
5	16	0.1526	0.1190	0.0964	1.283	2.49
5	64	0.1410	0.1158	0.0964	1.218	2.30
5	256	0.1312	0.1225	0.0964	1.071	1.33
10	4	0.2209	0.1701	0.1102	1.299	1.85
10	16	0.1944	0.1484	0.1102	1.310	2.20
10	64	0.1700	0.1448	0.1102	1.174	1.73
10	256	0.1638	0.1622	0.1102	1.010	1.03
20	4	0.3766	0.2683	0.1263	1.404	1.76
20	16	0.2759	0.1983	0.1263	1.392	2.08
20	64	0.2104	0.1802	0.1263	1.168	1.56
20	256	0.2407	0.2328	0.1263	1.034	1.07
30	4	0.6366	0.5135	0.2188	1.240	1.42
30	16	0.4524	0.3362	0.2188	1.346	1.99
30	64	0.3806	0.3354	0.2188	1.135	1.39
30	256	0.3681	0.3569	0.2188	1.031	1.08

Table B.2: Batch-size sweep at $p = 20$: final loss and excess over the floor for each estimator.

B	MC loss	RB loss	MC excess	RB excess	excess ratio
4	0.377	0.268	0.250	0.142	1.76
16	0.276	0.198	0.150	0.072	2.08
64	0.210	0.180	0.084	0.054	1.56
256	0.241	0.233	0.114	0.107	1.07

The Rao-Blackwellised estimator at batch 4 (excess 0.142) matches the Monte Carlo estimator at batch 16 (excess 0.150): its variance reduction is worth a fourfold increase in batch size. At every batch size its excess over the floor is the smaller of the two, and only at $B = 256$ (where both estimators have nearly exhausted the Monte Carlo noise) do the two nearly coincide.

B.4 Bayes-optimal floors

Table B.3 reports the Bayes floor for each dimension at $n = 50$, together with the two independent estimators of it: the average within-chain posterior variance and the mean squared error of the posterior-mean estimate (Section 4.2.4). The two agree to within sampling error at every dimension, validating the Gibbs sampler.

Table B.3: Bayes-optimal floors by dimension p (at $n = 50$), with the squared-error and variance-based estimators.

p	L^* (MSE)	L^* (variance)
2	0.0708	0.0774
5	0.0964	0.0976
10	0.1102	0.1131
20	0.1263	0.1276
30	0.2188	0.2165

B.5 Second design matrix

The main-text results fix a single design matrix X per dimension. To confirm that the Rao-Blackwellisation advantage is a property of the estimator and not of one particular design, the entire sweep was repeated on a second, independent design-matrix realisation $X^{(2)}$ (a different random seed for X ; identical prior, grid, ten training seeds, and 200 epochs). Its Bayes floors are recomputed by the same Gibbs sampler on $X^{(2)}$ and reported in the L^* column of Table B.4.

The pattern of the main text reappears (Figure B.1): the Rao-Blackwellised estimator attains the lower loss in eighteen of the twenty cells, by margins that grow as the batch size shrinks. The two exceptions are both at the largest batch $B = 256$, where the two estimators have all but exhausted the Monte Carlo noise and coincide to within 2% — the same vanishing of the Rao-Blackwell advantage with batch size seen for the first design. The advantage is therefore robust to the design wherever it is not already exhausted by averaging.

B.6 Reproduction

The experiment code is in `sweeps2/` of the accompanying repository. The training curves of Figure 4.1 and the raw per-cell trajectories are produced by

```
python rb-vs.mc-sweep-tau.py
```

and the Bayes floors and the tables of this appendix by `floors_and_tables.py`, which runs a vectorised Gibbs sampler for the linear-regression posterior on the same held-out test sets.

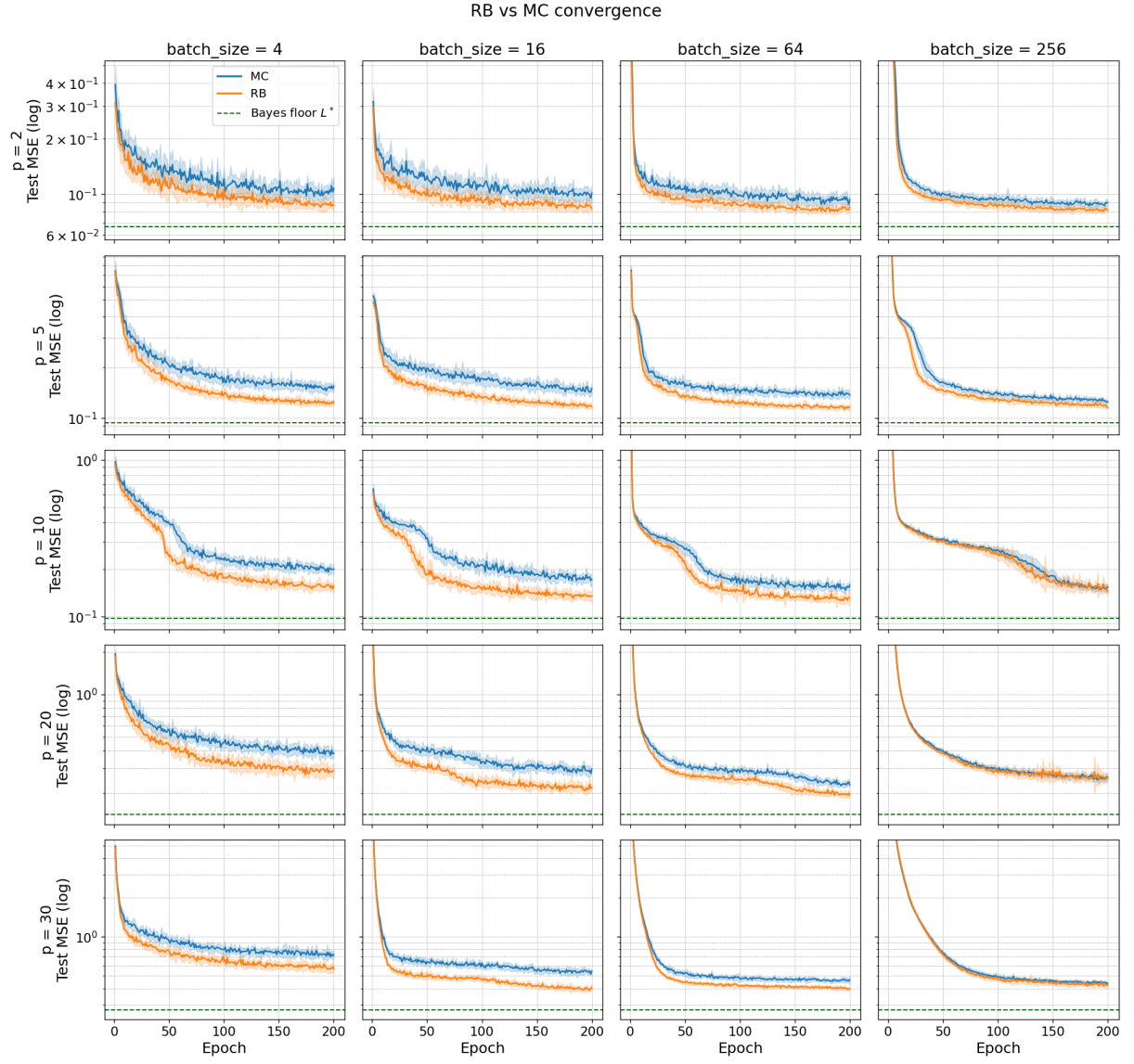


Figure B.1: As Figure 4.1, for the second design-matrix realisation $X^{(2)}$: training curves across parameter dimension p (rows) and batch size (columns); Monte Carlo (blue) and Rao-Blackwellised (orange), with the Bayes floor $L^*(p)$ dashed.

Table B.4: Second design matrix $X^{(2)}$: final test loss of the Monte Carlo (MC) and Rao-Blackwellised (RB) estimators, the Bayes floor L^* , and the raw and excess-over-floor ratios, for each (p, B) cell at $n = 50$.

p	B	$\mathcal{L}_{\text{MC}}$	$\mathcal{L}_{\text{RB}}$	L^*	ratio	excess ratio
2	4	0.1041	0.0882	0.0666	1.180	1.74
2	16	0.1001	0.0861	0.0666	1.162	1.72
2	64	0.0936	0.0838	0.0666	1.117	1.57
2	256	0.0889	0.0823	0.0666	1.080	1.42
5	4	0.1508	0.1234	0.0939	1.222	1.93
5	16	0.1471	0.1187	0.0939	1.240	2.15
5	64	0.1396	0.1160	0.0939	1.204	2.07
5	256	0.1271	0.1187	0.0939	1.070	1.34
10	4	0.2002	0.1561	0.0972	1.282	1.75
10	16	0.1762	0.1360	0.0972	1.295	2.04
10	64	0.1540	0.1294	0.0972	1.189	1.76
10	256	0.1524	0.1531	0.0972	0.995	0.99
20	4	0.3888	0.2864	0.1407	1.357	1.70
20	16	0.2912	0.2156	0.1407	1.351	2.01
20	64	0.2311	0.1953	0.1407	1.183	1.66
20	256	0.2538	0.2588	0.1407	0.981	0.96
30	4	0.7288	0.5779	0.2737	1.261	1.50
30	16	0.5409	0.3974	0.2737	1.361	2.16
30	64	0.4639	0.4018	0.2737	1.155	1.49
30	256	0.4435	0.4278	0.2737	1.037	1.10

The Bayes-floor line on Figure 4.1 is overlaid by `replot_curves_floor.py`, and the samples-seen view of Figure 4.2 by `plot_curves_vs_samples.py`; both reuse the saved `.npz` without retraining.

Bibliography

- [1] James O Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer, 2 edition, 1985.
- [2] Michael Betancourt. A conceptual introduction to hamiltonian monte carlo. *arXiv preprint arXiv:1701.02434*, 2017.
- [3] Steve Brooks, Andrew Gelman, Galin L Jones, and Xiao-Li Meng, editors. *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, 2011.
- [4] Bob Carpenter, Andrew Gelman, Matthew D Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017.
- [5] George Casella and Edward I George. Explaining the gibbs sampler. *The American Statistician*, 46(3):167–174, 1992.
- [6] Siddhartha Chib and Edward Greenberg. Understanding the metropolis–hastings algorithm. *The American Statistician*, 49(4):327–335, 1995.
- [7] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [8] Kyle Cranmer, Johann Brehmer, and Gilles Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, 2020.
- [9] Andrew Gelman, John B Carlin, Hal S Stern, David B Dunson, Aki Vehtari, and Donald B Rubin. *Bayesian Data Analysis*. CRC Press, 3 edition, 2013.
- [10] Stuart Geman and Donald Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6:721–741, 1984.
- [11] Geof H Givens and Jennifer A Hoeting. *Computational Statistics*. Wiley, 2 edition, 2013.
- [12] W Keith Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970.

- [13] Matthew D Hoffman and Andrew Gelman. The no-u-turn sampler: adaptively setting path lengths in hamiltonian monte carlo. *Journal of Machine Learning Research*, 15(1):1593–1623, 2014.
- [14] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [15] Erich L Lehmann and George Casella. *Theory of Point Estimation*. Springer, 2 edition, 1998.
- [16] Jan-Matthis Lueckmann, Jan Boelts, David S Greenberg, Pedro J Gonçalves, and Jakob H Macke. Benchmarking simulation-based inference. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 130 of *PMLR*, pages 343–351, 2021.
- [17] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.
- [18] Radford M Neal. Mcmc using hamiltonian dynamics. In *Handbook of Markov Chain Monte Carlo*, pages 113–162. CRC Press, 2011.
- [19] George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. In *Advances in Neural Information Processing Systems*, 2017.
- [20] George Papamakarios, David Sterratt, and Iain Murray. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *International Conference on Artificial Intelligence and Statistics*, 2019.
- [21] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.
- [22] Thomas P Prescott, David J Warne, and Ruth E Baker. Efficient multifidelity likelihood-free Bayesian inference with adaptive computational resource allocation. *Journal of Computational Physics*, 496:112577, 2024.
- [23] Stefan T Radev, Ulf K Mertens, Andreas Voss, Lynton Ardizzone, and Ullrich Köthe. BayesFlow: Learning complex stochastic models with invertible neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 33(4):1452–1466, 2022.
- [24] Stefan T Radev, Marvin Schmitt, Lukas Schumacher, Lasse Elsemüller, Valentin Pratz, Yannik Schälte, Ullrich Köthe, and Paul-Christian Bürkner. BayesFlow: Amortized Bayesian workflows with neural networks. *Journal of Open Source Software*, 8(89):5702, 2023.
- [25] Christian P Robert and George Casella. *Monte Carlo Statistical Methods*. Springer, 2 edition, 2004.
- [26] Matthew Sainsbury-Dale, Andrew Zammit-Mangion, and Raphaël Huser. Likelihood-free

- parameter estimation with neural bayes estimators. *The American Statistician*, 78(1):1–14, 2024.
- [27] Matthew Sainsbury-Dale, Andrew Zammit-Mangion, Jordan Richards, and Raphaël Huser. Neural bayes estimators for irregular spatial data using graph neural networks. *Journal of Computational and Graphical Statistics*, 2025.
 - [28] Chong Wang, Xi Chen, Alexander J Smola, and Eric P Xing. Variance reduction for stochastic gradient optimization. In *Advances in Neural Information Processing Systems*, 2013.
 - [29] David J Warne, Thomas P Prescott, Ruth E Baker, and Matthew J Simpson. Multifidelity multilevel Monte Carlo to accelerate approximate Bayesian parameter inference for partially observed stochastic processes. *Journal of Computational Physics*, 469:111543, 2022.
 - [30] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. Deep sets. In *Advances in Neural Information Processing Systems*, pages 3391–3401, 2017.
 - [31] Andrew Zammit-Mangion, Matthew Sainsbury-Dale, and Raphaël Huser. Neural methods for amortized inference. *Annual Review of Statistics and Its Application*, 12:311–335, 2025.